

**POSTS AND TELECOMMUNICATIONS INSTITUTE
OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY 1**



**SOFTWARE ARCHITECTURE
AND DESIGN
Báo cáo Tiểu luận V2**

Lecturer: Trần Đình Quế
Member: B22DCDT059 - Trần Anh Dũng
Class: E22CNPM01

Hà Nội, 2026

Mục lục

1	CHƯƠNG 1: Từ Monolithic đến Microservices và DDD	7
1.1	Giới thiệu Monolithic Architecture	7
1.1.1	Khái niệm	7
1.1.2	Cấu trúc điển hình	7
1.1.3	Ví dụ thực tế	7
1.1.4	Nhược điểm chi tiết	7
1.1.5	Khi nào nên dùng Monolithic	7
1.2	Microservices Architecture	8
1.2.1	Khái niệm	8
1.2.2	Đặc điểm	8
1.2.3	So sánh Monolithic vs Microservices	8
1.2.4	Ưu điểm	9
1.2.5	Nhược điểm	9
1.2.6	Nguyên tắc thiết kế	9
1.3	Nghiên cứu ứng dụng Microservices tại các Big Tech	9
1.4	Triển khai Microservices sử dụng framework Django	10
1.5	Domain Driven Design (DDD)	10
1.5.1	Mục tiêu	10
1.5.2	Các khái niệm cốt lõi	11
1.5.3	Context Map	11
1.5.4	DDD trong Microservices	11
1.6	Case Study: Healthcare (Luyện Decomposition)	11
1.6.1	Mô tả bài toán	11
1.6.2	Thiết kế hệ thống theo hướng Monolithic	11
1.6.3	Thiết kế hệ thống theo hướng Microservices (Áp dụng DDD)	12
1.6.4	Bước 1: Xác định Domain	12
1.6.5	Bước 2: Xác định Bounded Context	13
1.6.6	Bước 3: Quan hệ giữa các Context	13

1.7	Kết luận	13
2	CHƯƠNG 2: Phát triển Hệ E-Commerce Microservices	14
2.1	Xác định yêu cầu	14
2.1.1	Functional Requirements	14
2.1.2	Non-functional Requirements	14
2.1.3	Biểu đồ Usecase (Usecase Diagram)	14
2.2	Phân rã hệ thống theo DDD	16
2.2.1	Bounded Context	16
2.2.2	Nguyên tắc	16
2.3	Thiết kế Product Service – <code>pharmacy-service + medical-catalog-service</code>	16
2.3.1	Phân loại sản phẩm	16
2.3.2	Model tổng quát	17
2.3.3	Chi tiết theo domain	17
2.3.4	API	17
2.4	Thiết kế User Service – <code>auth-service + patient-service</code>	18
2.4.1	Phân loại người dùng	18
2.4.2	Model	18
2.4.3	Phân quyền (RBAC)	19
2.4.4	API	19
2.5	Thiết kế Cart Service – <code>prescription-service</code>	19
2.5.1	Model	19
2.5.2	Logic	19
2.5.3	API	20
2.6	Thiết kế Order Service – <code>dispensing-service</code>	20
2.6.1	Model	20
2.6.2	Workflow	21
2.7	Thiết kế Payment Service – tích hợp trong <code>dispensing-service</code>	21
2.7.1	Model	21
2.7.2	Trạng thái	21

2.7.3	API	22
2.8	Thiết kế Shipping Service – tích hợp trong <code>dispensing-service</code>	22
2.8.1	Model	22
2.8.2	Trạng thái	22
2.8.3	API	22
2.9	Luồng hệ thống tổng thể	22
2.10	Hướng dẫn thực hành	23
2.10.1	Mục tiêu	23
2.10.2	Sơ đồ kiến trúc Microservice và Class Diagram	23
2.11	Biểu đồ luồng xử lý quan trọng – Quy trình Mua Hàng	29
2.11.1	Mapping Class Diagram sang Database	31
2.11.2	Data Model (ERD) và Database Schema từng Service	31
2.11.3	So sánh MySQL vs PostgreSQL	35
2.11.4	Bài tập	35
2.11.5	Checklist đánh giá	35
2.12	Kết luận	35
3	CHƯƠNG 3: AI Service cho tư vấn sản phẩm	36
3.1	Mục tiêu	36
3.2	Kiến trúc AI Service	36
3.3	Thu thập dữ liệu	36
3.3.1	User Behavior Data	36
3.3.2	Nguồn dữ liệu thực nghiệm (Dataset)	37
3.3.3	Ví dụ dataset hành vi (User Behavior Data)	37
3.4	Mô hình LSTM (Sequence Modeling)	37
3.4.1	Ý tưởng	37
3.4.2	Model chi tiết	37
3.4.3	Training	38
3.5	Knowledge Graph với Neo4j	38
3.5.1	Mô hình đồ thị	38

3.5.2	Ví dụ Cypher	38
3.5.3	Truy vấn gợi ý	38
3.6	RAG (Retrieval-Augmented Generation)	38
3.6.1	Pipeline	38
3.6.2	Vector Database	39
3.6.3	Ví dụ	39
3.7	Kết hợp Hybrid Model	39
3.8	Đánh giá và So sánh Mô hình	39
3.9	Hai dạng AI Service	41
3.9.1	1. Recommendation List	41
3.9.2	2. Chatbot tư vấn	41
3.10	Triển khai AI Service	42
3.10.1	Tech stack thực tế	42
3.10.2	Sơ đồ kiến trúc và triển khai AI Service	43
3.10.3	Thành phần 1 – Deep Learning (GNN + SPD-AIRM)	43
3.10.4	Thành phần 2 – Knowledge Graph (Neo4j)	44
3.10.5	Thành phần 3 – RAG Pipeline (FAISS + Gemini LLM)	44
3.11	Bài tập	44
3.12	Checklist đánh giá	45
3.13	Kết luận	45
4	CHƯƠNG 4: Xây dựng hệ thống hoàn chỉnh	46
4.1	Kiến trúc tổng thể	46
4.1.1	Mô hình hệ thống	46
4.1.2	Nguyên tắc	46
4.2	System Architecture	46
4.2.1	Overview	46
4.2.2	Microservice Architecture	47
4.2.3	API Gateway	47
4.2.4	Service Communication	47

4.2.5	Containerization and Deployment	47
4.2.6	System Structure	48
4.2.7	Design Principles	48
4.2.8	Security Considerations	48
4.2.9	Discussion	48
4.3	API Gateway (Nginx)	49
4.3.1	Vai trò	49
4.3.2	Cấu hình mẫu	49
4.4	Authentication (JWT)	49
4.4.1	Cài đặt	49
4.4.2	Cấu hình	49
4.4.3	Luồng	49
4.5	Giao tiếp giữa các Service	49
4.5.1	REST API call	49
4.5.2	Best Practice	50
4.6	Docker hóa hệ thống	50
4.6.1	Dockerfile (Django)	50
4.6.2	docker-compose.yml	50
4.7	Luồng hệ thống (End-to-End)	50
4.7.1	Use case: Mua hàng	50
4.7.2	Sequence logic	51
4.8	Triển khai Kubernetes (Optional)	51
4.8.1	Deployment	51
4.8.2	Service	51
4.9	Logging và Monitoring	51
4.10	Đánh giá hệ thống	51
4.10.1	Hiệu năng	51
4.10.2	Khả năng mở rộng	52
4.10.3	Ưu điểm	52

4.10.4	Nhược điểm	52
4.11	Kết quả thực thi và Giao diện hệ thống	52
4.12	Code Logic Core – Trích Xuất tu Du an Thuc Te	55
4.12.1	API Tao Don Hang – dispensing-service	55
4.12.2	Application Layer – CreateOrderCommandHandler	55
4.12.3	AI Chatbot GraphRAG – ai-service/main.py	56
4.12.4	AI Intent Classifier – services/ai_logic.py	56
4.12.5	Recommendation API – GNN + SPD-AIRM Manifold	57
4.13	Checklist đánh giá	57
Kết luận		58

1 CHƯƠNG 1: Từ Monolithic đến Microservices và DDD

1.1 Giới thiệu Monolithic Architecture

1.1.1 Khái niệm

Monolithic Architecture là mô hình trong đó toàn bộ hệ thống (UI, business logic, database access) được xây dựng và triển khai như một khối duy nhất. Tất cả các module được đóng gói chung trong một process hoặc ứng dụng.

1.1.2 Cấu trúc điển hình

- **Presentation Layer (UI):** Chịu trách nhiệm hiển thị giao diện.
- **Business Logic Layer:** Xử lý nghiệp vụ chính của hệ thống.
- **Data Access Layer:** Thao tác với cơ sở dữ liệu.

1.1.3 Ví dụ thực tế

Một hệ thống e-commerce y tế theo kiến trúc monolithic sẽ bao gồm tất cả các thành phần sau trong cùng một codebase: Quản lý sản phẩm, Giỏ hàng, Thanh toán, và User.

1.1.4 Nhược điểm chi tiết

- **Khó mở rộng:** Phải scale toàn bộ hệ thống thay vì từng module độc lập.
- **Coupling cao:** Một thay đổi nhỏ ở module này có thể ảnh hưởng lớn đến toàn bộ hệ thống.
- **Deploy rủi ro:** Bất cứ một lỗi nhỏ nào cũng có thể làm sập toàn bộ hệ thống.
- **Khó phát triển nhóm:** Quá nhiều team cùng sửa chung một codebase dẫn đến conflict liên tục.

1.1.5 Khi nào nên dùng Monolithic

- Hệ thống có quy mô nhỏ, nghiệp vụ đơn giản.
- Xây dựng MVP (Minimum Viable Product) để kiểm chứng ý tưởng.
- Team phát triển ít người, chưa có kinh nghiệm về phân tán.

1.2 Microservices Architecture

1.2.1 Khái niệm

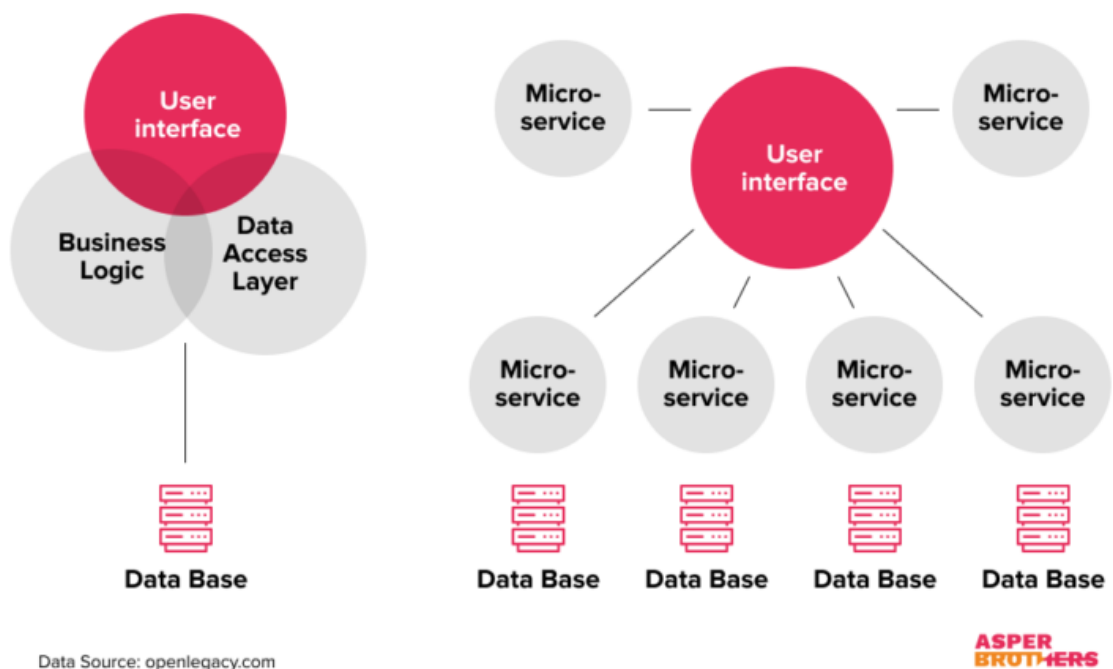
Microservices là kiến trúc chia hệ thống thành các dịch vụ (services) nhỏ, hoạt động độc lập. Mỗi service tập trung thực hiện một chức năng nghiệp vụ riêng biệt.

1.2.2 Đặc điểm

- Mỗi service sở hữu database riêng (Database-per-service).
- Các service giao tiếp với nhau qua API (REST, gRPC) hoặc Message Broker.
- Mỗi service được deploy, scale và bảo trì một cách hoàn toàn độc lập.

1.2.3 So sánh Monolithic vs Microservices

MICROSERVICES VS MONOLITHIC ARCHITECTURE



Hình 1.1: So sánh kiến trúc Monolithic và Microservices

Tiêu chí	Monolithic	Microservices
Deploy	Một lần cho toàn bộ ứng dụng	Nhiều lần, độc lập từng service
Scale	Phải scale toàn bộ hệ thống	Scale độc lập từng service
Coupling	Cao – một thay đổi ảnh hưởng cả hệ thống	Thấp – các service hoàn toàn tách biệt
Database	Dùng chung một DB lớn	Database-per-service, hoàn toàn độc lập
Fault isolation	Lỗi 1 module sập cả hệ thống	Lỗi 1 service không ảnh hưởng service khác
Phát triển nhóm	Nhiều team tranh chung codebase	Mỗi team sở hữu 1 service riêng

Bảng 1: Bảng so sánh chi tiết Monolithic vs Microservices

1.2.4 Ưu điểm

- Khả năng mở rộng (Scale) độc lập, tối ưu hóa tài nguyên.
- Tăng tốc độ phát triển nhờ việc các team có thể làm việc song song.
- Rất phù hợp với các hệ thống có quy mô lớn và phức tạp.

1.2.5 Nhược điểm

- Độ phức tạp hệ thống tăng lên rất nhiều.
- Khó khăn trong việc quản lý, giám sát (distributed system).
- Khó debug và rà soát lỗi liên service.

1.2.6 Nguyên tắc thiết kế

- **Single Responsibility:** Mỗi service chỉ đảm nhận một trách nhiệm duy nhất.
- **Loose Coupling:** Giảm thiểu sự phụ thuộc giữa các service.
- **High Cohesion:** Gom nhóm các thành phần có liên quan chặt chẽ vào cùng một service.

1.3 Nghiên cứu ứng dụng Microservices tại các Big Tech

Các tập đoàn công nghệ lớn (Big Tech) đã đi đầu trong việc chuyển đổi kiến trúc từ Monolithic sang Microservices để giải quyết bài toán về khả năng mở rộng và tốc độ phát triển.

- **Netflix:** Chuyển đổi từ năm 2009 sau một sự cố sập server lớn. Họ chia nhỏ hệ thống thành hàng trăm vi dịch vụ độc lập, giúp xử lý hàng tỷ lượt streaming mỗi ngày và đảm bảo tính sẵn sàng cao (High Availability).
- **Amazon:** Đối mặt với mã nguồn monolithic quá lớn vào đầu những năm 2000, Amazon đã tái cấu trúc thành kiến trúc SOA (Service-Oriented Architecture), tiền thân của Microservices. Mỗi nhóm phát triển một service độc lập, dẫn đến sự ra đời của AWS.
- **Uber:** Ban đầu xây dựng với kiến trúc nguyên khối cho một thành phố, khi mở rộng toàn cầu, họ đã chuyển sang kiến trúc vi dịch vụ (mỗi chức năng như điều phối xe, thanh toán, đánh giá là một service riêng) để dễ dàng scale và thêm tính năng mới.

1.4 Triển khai Microservices sử dụng framework Django

Trong dự án này, hệ thống được triển khai theo kiến trúc Microservices sử dụng **Django** (kết hợp với FastAPI). Lựa chọn này dựa trên các yếu tố:

- **Tốc độ phát triển (Rapid Development):** Django cung cấp sẵn các ORM mạnh mẽ, hệ thống quản lý admin và bảo mật, giúp xây dựng nhanh các service cốt lõi (như Product, Order, User).
- **Hỗ trợ hệ sinh thái AI:** Ngôn ngữ Python là nền tảng số một cho AI/ML. Việc sử dụng Django giúp các Microservice dễ dàng giao tiếp và tích hợp chung với AI Service.
- **Tính độc lập (Decoupling):** Mỗi Django project đóng vai trò là một service riêng, giao tiếp qua REST API (sử dụng Django REST Framework), có database riêng biệt (PostgreSQL).

1.5 Domain Driven Design (DDD)

1.5.1 Mục tiêu

DDD giúp mô hình hóa hệ thống phần mềm dựa trên nghiệp vụ thực tế (business domain), giúp cho thiết kế phần mềm độc lập và không bị phụ thuộc quá nhiều vào công nghệ bên dưới.

1.5.2 Các khái niệm cốt lõi

- **Entity:** Đối tượng có định danh (ID) duy nhất. Ví dụ: `User`, `Product`.
- **Value Object:** Đối tượng không có ID, được so sánh bằng giá trị của nó. Ví dụ: `Address`, `Money`.
- **Aggregate:** Nhóm các Entity và Value Object liên quan lại với nhau thành một đơn vị. Ví dụ: `Order` bao gồm `OrderItem`.
- **Bounded Context:** Ranh giới rõ ràng của một domain. Ví dụ: `User Context`, `Order Context`.

1.5.3 Context Map

Context Map thể hiện mối quan hệ giữa các Bounded Context. Các mô hình quan hệ phổ biến:

- **Shared Kernel:** Chia sẻ một phần mô hình chung.
- **Customer-Supplier:** Một context đóng vai trò cung cấp dữ liệu cho context khác.

1.5.4 DDD trong Microservices

- Áp dụng nguyên tắc: Mỗi Bounded Context ánh xạ thành 1 microservice.
- Tránh phân rã hệ thống theo cấu trúc technical layer (như chia service theo UI, Database).

1.6 Case Study: Healthcare (Luyện Decomposition)

1.6.1 Mô tả bài toán

Dự án e-commerce y tế yêu cầu quản lý thông tin bệnh nhân, danh mục thuốc, giỏ hàng, đặt hàng, thanh toán và một hệ thống AI tư vấn y tế. Việc thiết kế và phân rã hệ thống hợp lý là bắt buộc.

1.6.2 Thiết kế hệ thống theo hướng Monolithic

Ban đầu, hệ thống có thể được thiết kế theo hướng nguyên khối (Monolithic) để dễ dàng triển khai ở giai đoạn đầu (MVP). Trong mô hình này, toàn bộ UI, Backend Logic

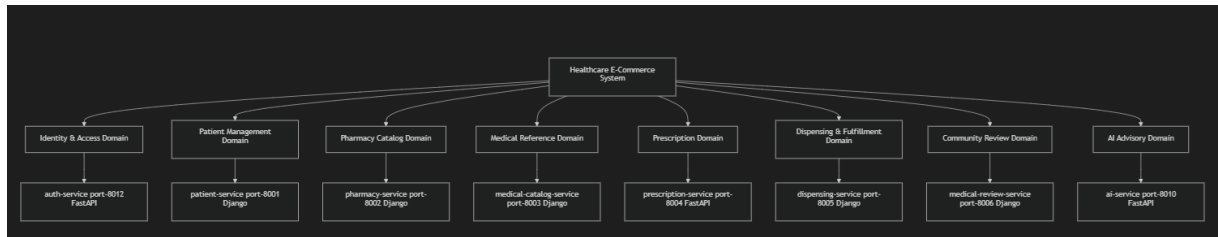
(Patient, Product, Order, Chatbot) và Database được gộp chung vào một khối (process) duy nhất.

- **Ưu điểm:** Dễ deploy, luồng dữ liệu nội bộ không cần gọi qua mạng (Network Calls).
- **Nhược điểm:** Khi tích hợp module AI (tốn nhiều CPU/RAM) hoặc lượng user mua thuốc tăng đột biến, toàn bộ hệ thống sẽ gặp hiện tượng nghẽn thắt cổ chai (bottleneck) và sập do không thể scale riêng lẻ.

Hình 1.2: Thiết kế hệ thống Healthcare E-Commerce theo kiến trúc Monolithic

1.6.3 Thiết kế hệ thống theo hướng Microservices (Áp dụng DDD)

Để giải quyết nhược điểm của Monolithic, ta thực hiện phân rã hệ thống thành các service nhỏ độc lập dựa trên nghiệp vụ.



Hình 1.3: Phân rã hệ thống Healthcare E-Commerce thành 8 Bounded Context và Microservices tương ứng

1.6.4 Bước 1: Xác định Domain

Xác định các domain cốt lõi của hệ thống Healthcare:

- **Identity & Access:** Quản lý xác thực, phân quyền người dùng.
- **Patient Management:** Hồ sơ bệnh nhân, bảo hiểm y tế.
- **Pharmacy Catalog:** Danh mục thuốc, quản lý kho.
- **Prescription:** Đơn thuốc tạm (tương đương giỏ hàng).
- **Dispensing & Fulfillment:** Đặt hàng, thanh toán, xuất thuốc.
- **Medical AI Advisory:** Chatbot GraphRAG + gợi ý sản phẩm.

1.6.5 Bước 2: Xác định Bounded Context

Dựa trên Domain, xác định 8 Bounded Context ánh xạ thành 8 Microservice:

- **Identity & Access Context** → `auth-service` (:8012)
- **Patient Management Context** → `patient-service` (:8001)
- **Pharmacy Catalog Context** → `pharmacy-service` (:8002)
- **Medical Reference Context** → `medical-catalog-service` (:8003)
- **Prescription Context** → `prescription-service` (:8004)
- **Dispensing Context** → `dispensing-service` (:8005)
- **Community Review Context** → `medical-review-service` (:8006)
- **AI Advisory Context** → `ai-service` (:8010)

1.6.6 Bước 3: Quan hệ giữa các Context

- `dispensing-service` phụ thuộc vào `prescription-service` (lấy giỏ hàng) và `patient-service` (kiểm tra bảo hiểm).
- `ai-service` truy vấn `pharmacy-service` để lấy danh sách sản phẩm cho gợi ý.
- Sau khi đặt hàng thành công, `dispensing-service` publish event `OrderFulfilled` → `pharmacy-service` trừ tồn kho.
- Giao tiếp nội bộ qua REST API; event-driven qua Redis message queue.

1.7 Kết luận

- Monolithic phù hợp với các hệ thống nhỏ.
- Microservices là giải pháp tối ưu cho hệ thống lớn và phức tạp.
- Áp dụng DDD là nền tảng cốt lõi để phân rã hệ thống một cách chính xác, tránh tạo ra Distributed Monolith.

2 CHƯƠNG 2: Phát triển Hệ E-Commerce Microservices

2.1 Xác định yêu cầu

2.1.1 Functional Requirements

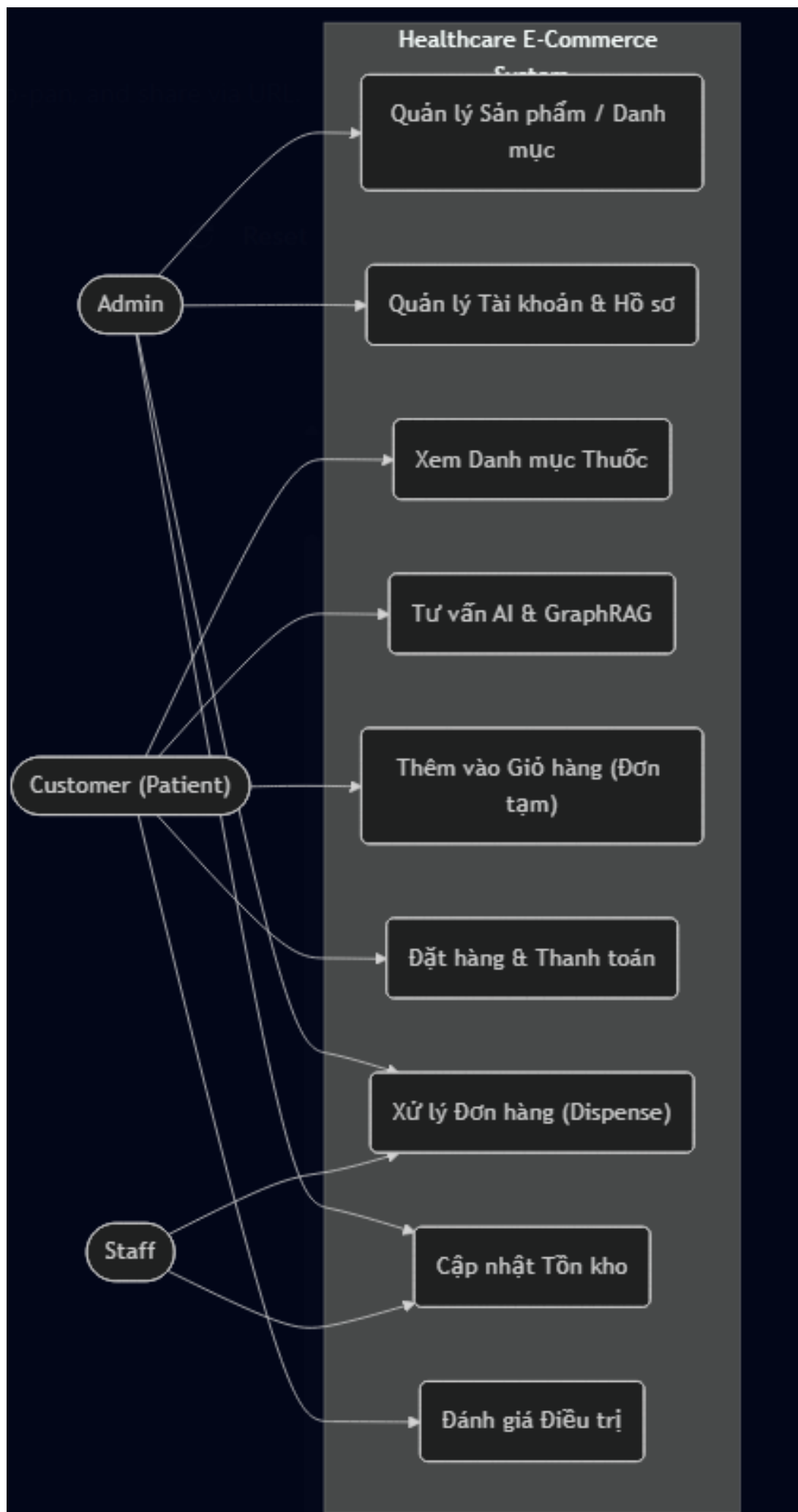
- **Quản lý sản phẩm y tế:** Thuốc (kê đơn / OTC), thiết bị y tế, thực phẩm chức năng – triển khai trong `pharmacy-service` và `medical-catalog-service`.
- **Quản lý người dùng:** Xác thực JWT (`auth-service`) và hồ sơ bệnh nhân (`patient-service`) theo 3 vai trò: Admin, Staff, Customer.
- **Giỏ hàng / Đơn thuốc tạm:** Khách hàng thêm/xóa sản phẩm qua `prescription-service` (tương đương Cart Service trong mẫu).
- **Đặt hàng và xuất thuốc:** Toàn bộ luồng Order – Payment – Shipping được xử lý tập trung trong `dispensing-service`.
- **Tìm kiếm và gợi ý sản phẩm AI:** Chatbot GraphRAG + Collaborative Filtering trong `ai-service`.
- **Đánh giá cộng đồng:** Bệnh nhân đánh giá hiệu quả điều trị qua `medical-review-service`.

2.1.2 Non-functional Requirements

- **Scalability:** Mỗi service scale độc lập theo traffic thực tế.
- **High Availability:** Lỗi một service không làm sập toàn hệ (fault isolation).
- **Security:** JWT HS256, RBAC, không expose port nội bộ ra ngoài.
- **Maintainability:** Kiến trúc 4-layer DDD, mỗi service có database và test riêng.
- **Performance:** Response time < 200ms cho REST, AI chat < 30s.

2.1.3 Biểu đồ Usecase (Usecase Diagram)

Biểu đồ Usecase mô tả các nhóm tác nhân (Admin, Staff, Customer) và các chức năng chính mà họ có thể tương tác trong hệ thống Healthcare.



2.2 Phân rã hệ thống theo DDD

2.2.1 Bounded Context

Thông qua phân tích Event Storming, hệ thống được phân rã thành 9 Bounded Context:

- **Identity & Access Context** → `auth-service (:8012)`
- **Patient Management Context** → `patient-service (:8001)`
- **Pharmacy Catalog Context** → `pharmacy-service (:8002)`
- **Medical Reference Context** → `medical-catalog-service (:8003)`
- **Prescription Context** → `prescription-service (:8004)`
- **Dispensing & Fulfillment Context** → `dispensing-service (:8005)`
- **Community Review Context** → `medical-review-service (:8006)`
- **AI Advisory Context** → `ai-service (:8010)`
- **API Gateway** → `api-gateway (:8000)`

2.2.2 Nguyên tắc

- Mỗi Bounded Context sở hữu database hoàn toàn độc lập (Database-per-Service).
- Giao tiếp giữa các context chỉ qua REST API hoặc Message Broker (RabbitMQ).
- Nghiêm cấm truy cập trực tiếp vào database của service khác.

2.3 Thiết kế Product Service – `pharmacy-service + medical-catalog-service`

(Trong mẫu giảng viên: *Product Service* ↔ Dự án: *`pharmacy-service :8002 Django + medical-catalog-service :8003 Django`*)

2.3.1 Phân loại sản phẩm

- **Thuốc kê đơn (Prescription):** Amoxicillin, Omeprazole, Nexium... (`requires_prescription`)
- **Thuốc không kê đơn (OTC):** Phosphalugel, Smecta, Domperidone...
- **Thực phẩm chức năng / Dược liệu:** Probiotics, herbal extract, vitamin.
- **Danh mục ICD-10:** Phân cấp theo `medical-catalog-service` (ví dụ: Bệnh tiêu hoá K00-K93).

2.3.2 Model tổng quát

```

1 # pharmacy-service/domain/product.py (Django Model)
2 class Product(models.Model):
3     product_code      = models.CharField(max_length=20, unique=True)
4     name              = models.CharField(max_length=255)
5     generic_name      = models.CharField(max_length=255)
6     category          = models.CharField(max_length=100)
7     dosage_form       = models.CharField(max_length=100)
8     price             = models.DecimalField(max_digits=12, decimal_places
9         =2)
10    stock_quantity    = models.IntegerField(default=0)
11    requires_prescription = models.BooleanField(default=False)
12    symptom_tags       = models.JSONField(default=list)
13    manufacturer      = models.CharField(max_length=255)
14
15 # medical-catalog-service/domain/category.py
16 class Category(models.Model):
17     name              = models.CharField(max_length=200)
18     slug              = models.SlugField(unique=True)
19     description       = models.TextField(blank=True)
20     parent            = models.ForeignKey('self', null=True, blank=True,
        on_delete=models.SET_NULL)

```

2.3.3 Chi tiết theo domain

Pharmacy Domain – Batch Tracking (Lô hàng):

```

1 class Batch(models.Model):
2     product          = models.ForeignKey(Product, on_delete=models.CASCADE)
3     batch_number     = models.CharField(max_length=50)
4     quantity         = models.IntegerField()
5     mfg_date         = models.DateField()
6     expiry_date      = models.DateField()

```

Medical Catalog Domain – Danh mục ICD-10:

```

1 # V    d    hierarchy:
2 # T i    u    h o    (K00-K93)
3 #           B n h    d    d    y    (K25-K31)
4 #           L o    t    d    d    y    t    t r    n g    (K25)

```

2.3.4 API

GET /products/ -- Danh sách thuốc (có filter, search)

POST /products/	-- Tạo sản phẩm mới (Admin)
GET /products/{id}/	-- Chi tiết sản phẩm
GET /categories/	-- Danh mục ICD-10
POST /inventory/adjust/	-- Cập nhật tồn kho (Staff)

2.4 Thiết kế User Service – auth-service + patient-service

(Trong mẫu giảng viên: User Service $\leftrightarrow$ Dự án: *auth-service :8012 FastAPI + patient-service :8001 Django*)

2.4.1 Phân loại người dùng

- **Admin:** Toàn quyền quản lý hệ thống, sản phẩm, và người dùng.
- **Staff:** Xử lý đơn thuốc, kiểm kho, cập nhật tồn kho.
- **Customer (Bệnh nhân):** Xem sản phẩm, tư vấn AI, đặt thuốc, thanh toán.

2.4.2 Model

```

1 # auth-service/domain/user.py (FastAPI + SQLAlchemy)
2 class User(Base):
3     __tablename__ = "users"
4     id = Column(Integer, primary_key=True)
5     email = Column(String, unique=True, nullable=False)
6     password_hash = Column(String, nullable=False)
7     fullname = Column(String)
8     is_active = Column(Boolean, default=True)
9     failed_attempts = Column(Integer, default=0)
10
11 # patient-service/domain/patient.py (Django)
12 class Patient(models.Model):
13     account_id = models.IntegerField(unique=True) # link to auth-
14         service
15     fullname = models.CharField(max_length=255)
16     blood_type = models.CharField(max_length=5)
17     insurance_code = models.CharField(max_length=50, blank=True)
18     total_spent = models.DecimalField(max_digits=12, decimal_places
19         =2)
20     membership_tier = models.CharField(max_length=10,
21         choices=[('BRONZE', 'Bronze'), ('SILVER', 'Silver'),
22             ('GOLD', 'Gold')])

```

2.4.3 Phân quyền (RBAC)

- **Admin:** CRUD toàn bộ tài nguyên hệ thống.
- **Staff:** Quản lý order và tồn kho; không có quyền quản lý user.
- **Customer:** Xem sản phẩm, đặt thuốc, xem lịch sử đơn hàng của chính mình.

2.4.4 API

POST /auth/register	-- Đăng ký tài khoản mới
POST /auth/login	-- Đăng nhập, trả về JWT token
POST /auth/validate	-- Xác thực token (nội bộ)
POST /auth/refresh	-- Làm mới access token
GET /patients/{id}/	-- Hồ sơ bệnh nhân
PUT /patients/{id}/	-- Cập nhật hồ sơ

2.5 Thiết kế Cart Service – prescription-service

(Trong mẫu giảng viên: Cart Service ↔ Dự án: *prescription-service* :8004 FastAPI)

2.5.1 Model

```
1 # prescription-service/domain/prescription.py (FastAPI + SQLAlchemy)
2 class CartItem(Base):
3     __tablename__ = "cart_items"
4     id = Column(Integer, primary_key=True)
5     customer_id = Column(String, nullable=False) # account_id
6     product_id = Column(Integer, nullable=False)
7     quantity = Column(Integer, default=1)
8     unit_price = Column(Numeric(12, 2))
9     product_name = Column(String)
10    created_at = Column(DateTime, default=datetime.utcnow)
```

2.5.2 Logic

- Thêm thuốc vào đơn tạm (Add to prescription basket).
- Cập nhật số lượng (Update quantity).
- Xóa thuốc khỏi đơn tạm (Remove item).
- Lấy toàn bộ đơn tạm theo customer_id (Get cart).

- Xóa toàn bộ sau khi dispensing-service tạo Order thành công (Clear after checkout).

2.5.3 API

```
POST /cart-items/          -- Thêm thuốc vào đơn tạm
GET  /cart-items/{cust_id}/ -- Xem đơn tạm hiện tại
PUT  /cart-items/{id}/     -- Cập nhật số lượng
DELETE /cart-items/{id}/   -- Xóa một dòng thuốc
DELETE /carts/{cust_id}/   -- Xóa toàn bộ giỏ (sau checkout)
```

2.6 Thiết kế Order Service – dispensing-service

(Trong mẫu giảng viên: Order Service ↔ Dự án: *dispensing-service* :8005 Django)

2.6.1 Model

```
1 # dispensing-service/domain/order.py (Django)
2 class Order(models.Model):
3     customer_id      = models.CharField(max_length=50)
4     status           = models.CharField(max_length=50,
5                                         choices=[('PENDING', 'Pending'), ('CONFIRMED', '
6                                             Confirmed'),
7                                             ('DISPENSED', 'Dispensed'), ('CANCELLED',
8                                             'Cancelled')])
9     payment_method    = models.CharField(max_length=50) # COD,
10    BANK_TRANSFER, BHYT
11    discount_rate      = models.DecimalField(max_digits=4, decimal_places
12    =2, default=0.0)
13    total_amount       = models.DecimalField(max_digits=12, decimal_places
14    =2)
15    created_at         = models.DateTimeField(auto_now_add=True)
16
17 class OrderItem(models.Model):
18     order             = models.ForeignKey(Order, on_delete=models.CASCADE)
19     product_id        = models.IntegerField()
20     name              = models.CharField(max_length=255)
21     price              = models.DecimalField(max_digits=12, decimal_places=2)
22     quantity          = models.IntegerField()
```

2.6.2 Workflow

- Tạo Order từ dữ liệu đơn tạm trong `prescription-service`.
- Áp dụng chiết khấu BHYT nếu bệnh nhân có `insurance_code` (`discount_rate = 0.2`).
- Sau khi tạo Order thành công: gọi `prescription-service` để xóa giỏ, gọi `patient-service` để cộng `total_spent` và tính lại `membership tier`.
- Publish `OrderFulfilled` event → RabbitMQ → `pharmacy-service` trừ tồn kho.

2.7 Thiết kế Payment Service – tích hợp trong `dispensing-service`

(Trong mẫu giảng viên: *Payment Service* ↔ Dự án: *Tích hợp sẵn trong `dispensing-service`*)

2.7.1 Model

```
1 # Trạng payment c nh ng tr c t i p trong b ng orders
2 # dispensing-service/domain/order.py
3 class Order(models.Model):
4     payment_method = models.CharField(max_length=50)
5     # COD: Thanh to n khi n h n h ng
6     # BANK_TRANSFER: Ch u y n k h o n ng n h ng
7     # BHYT: B o h i m y t (p d ng discount_rate = 20%)
8     discount_rate = models.DecimalField(max_digits=4, decimal_places=2,
9                                         default=0.0)
10    total_amount = models.DecimalField(max_digits=12, decimal_places
    =2)
11    # total = subtotal * (1 - discount_rate)
```

2.7.2 Trạng thái

- **PENDING:** Đơn hàng vừa được tạo, chờ xác nhận.
- **CONFIRMED:** Đơn hàng đã được Staff xác nhận.
- **DISPENSED:** Thuốc đã được phát (tương đương Delivered).
- **CANCELLED:** Đơn hàng bị hủy.

2.7.3 API

POST /orders/create/ -- Tạo đơn + tính tiền + áp dụng BHYT
 GET /orders/{id}/ -- Tra cứu trạng thái đơn hàng
 PUT /orders/{id}/status/ -- Cập nhật trạng thái (Staff/Admin)

2.8 Thiết kế Shipping Service – tích hợp trong dispensing-service

(Trong mẫu giảng viên: Shipping Service ↔ Dự án: Tích hợp trong *dispensing-service*)

2.8.1 Model

```

1 # Shipping c q u n l q u a t r n g shipping_method v status
   trong b n g orders
2 class Order(models.Model):
3     shipping_method = models.CharField(max_length=50,
4                                       choices=[('PICKUP', 'L y t i c a h n g'),
5                                                ('EXPRESS', 'Giao nhanh'),
6                                                ('STANDARD', 'Giao ti u c h u n')])
7     shipping_address = models.TextField(blank=True)
8     note = models.TextField(blank=True)

```

2.8.2 Trạng thái

- **PENDING → CONFIRMED → DISPENSED**: Luồng trạng thái đơn thuốc.
- Khi status = DISPENSED: hệ thống tự động cộng total_spent cho bệnh nhân và publish OrderFulfilled event.

2.8.3 API

GET /orders/{id}/ -- Xem trạng thái đơn hàng
 PUT /orders/{id}/status/ -- Cập nhật trạng thái giao hàng

2.9 Luồng hệ thống tổng thể

1. Người dùng đăng nhập qua api-gateway → JWT được cấp bởi auth-service.
2. Xem danh mục thuốc qua pharmacy-service; gợi ý cá nhân hoá từ ai-service.
3. Thêm thuốc vào đơn tạm qua prescription-service.

4. Thanh toán: `dispensing-service` lấy items từ `prescription-service`, tạo Order, áp BHYT.
5. `prescription-service` xóa giỏ; `patient-service` cập nhật membership tier.
6. RabbitMQ publish `OrderFulfilled` → `pharmacy-service` trừ tồn kho.
7. Bệnh nhân đánh giá qua `medical-review-service`; AI cập nhật behavior graph (Neo4j).

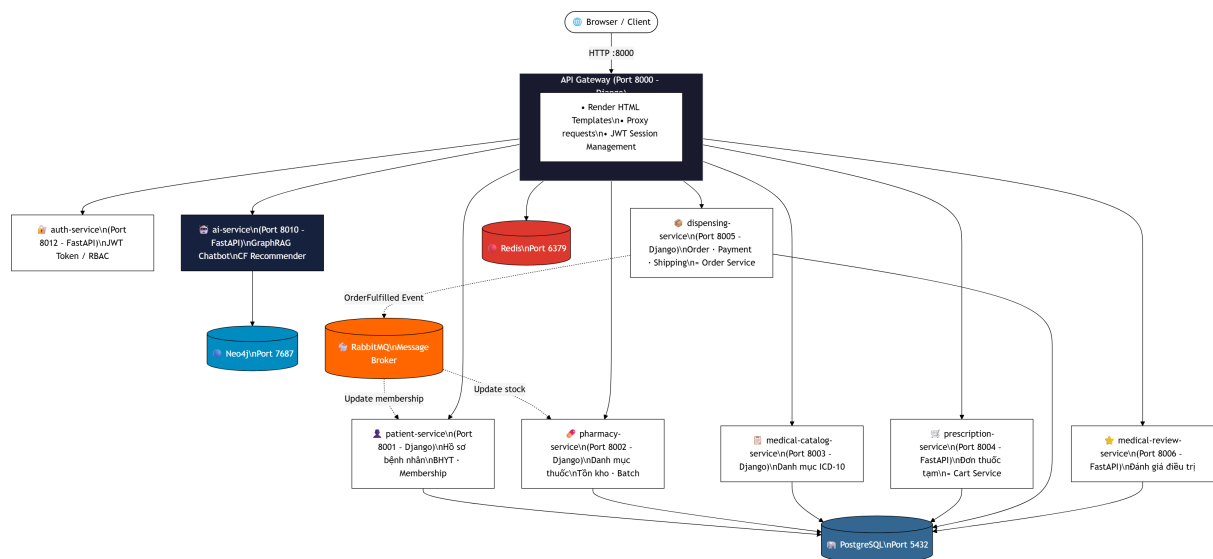
2.10 Hướng dẫn thực hành

2.10.1 Mục tiêu

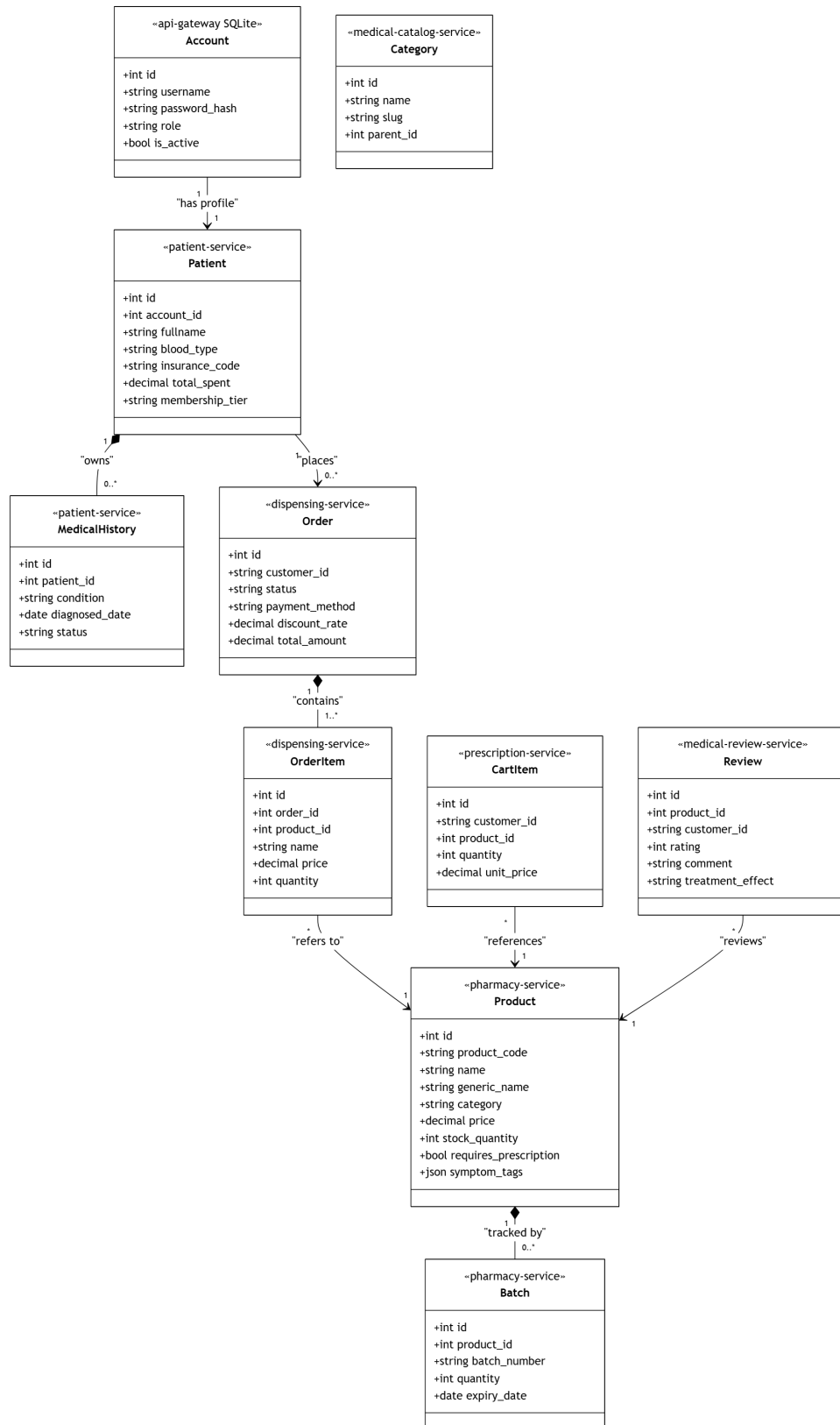
Sinh viên cần thực hiện ba nhiệm vụ cốt lõi:

- Thiết kế sơ đồ kiến trúc Microservice tổng thể.
- Vẽ Class Diagram cho toàn bộ hệ thống.
- Xây dựng và mapping từ Class Diagram sang database schema của từng service.

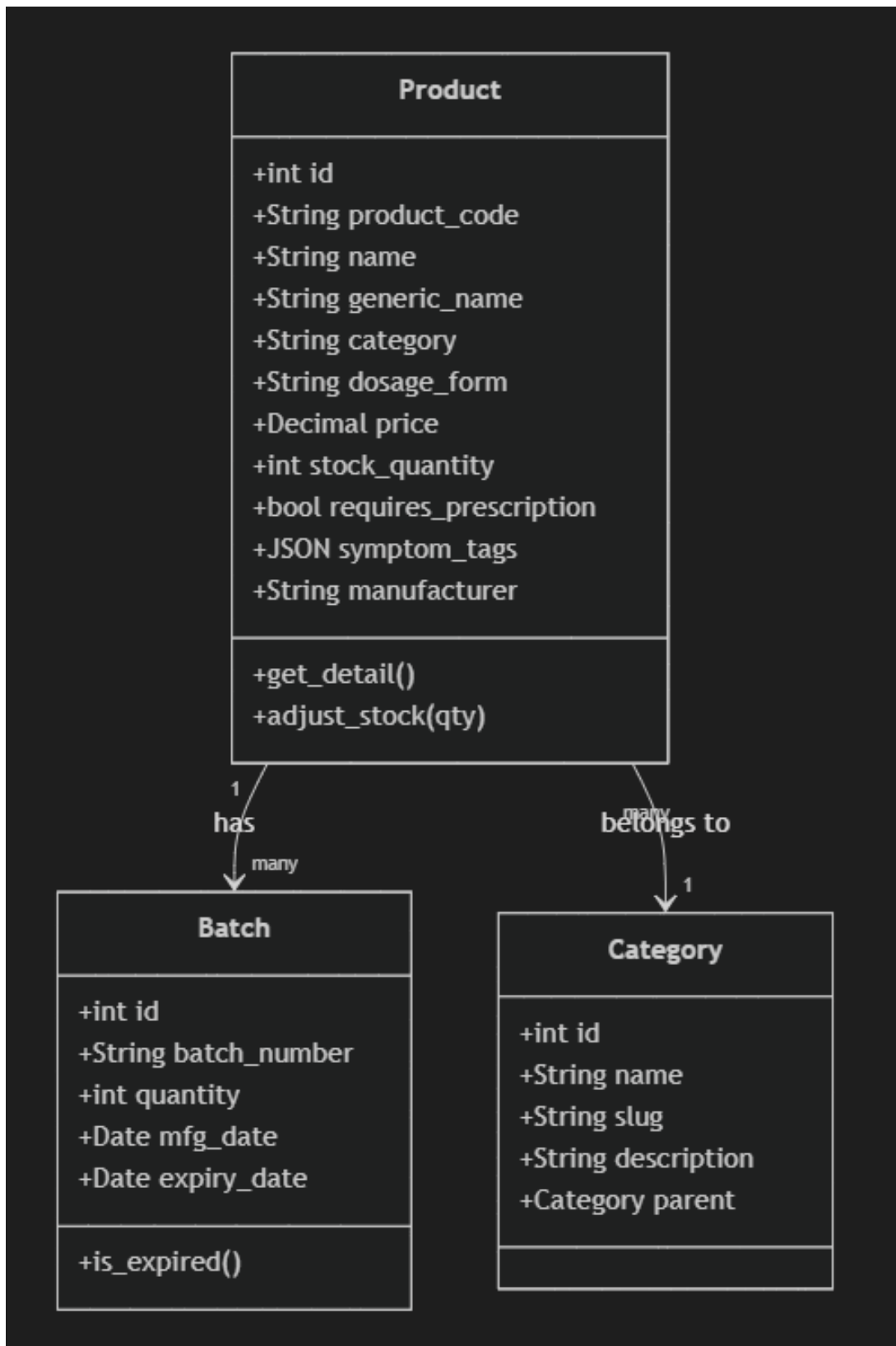
2.10.2 Sơ đồ kiến trúc Microservice và Class Diagram



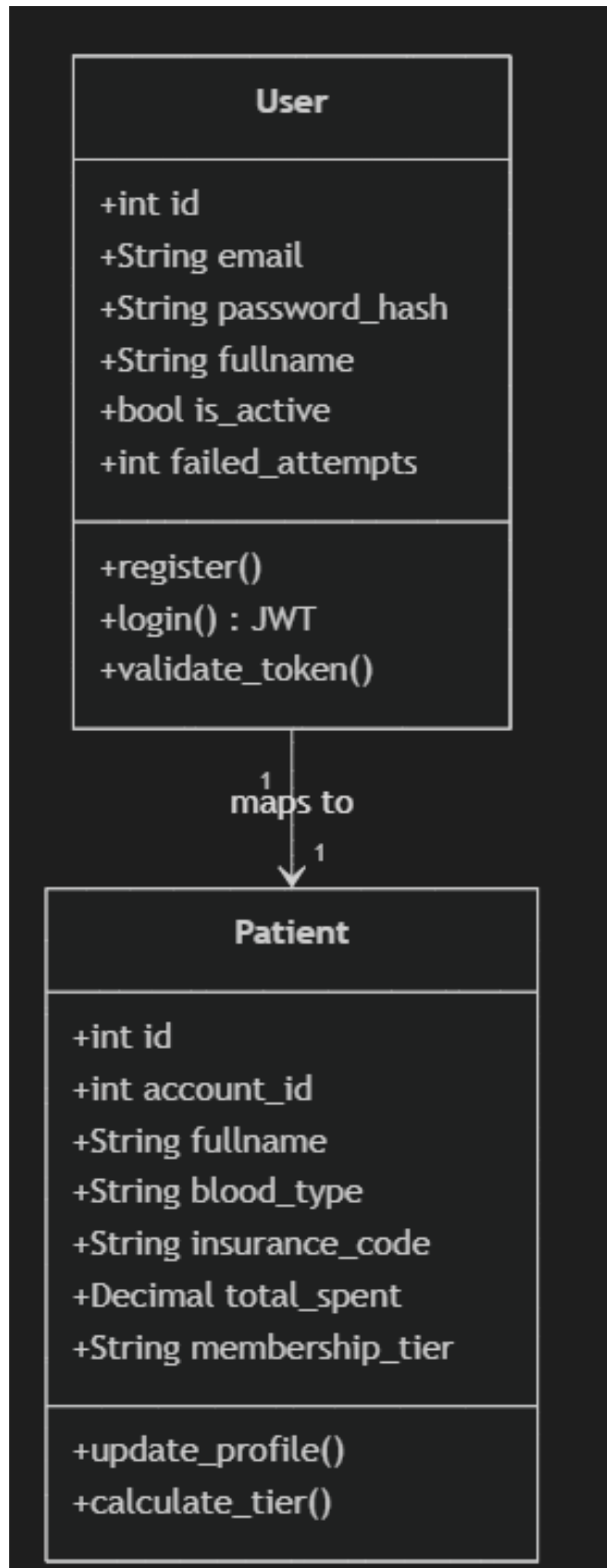
Hình 2.2: Sơ đồ kiến trúc Microservice hệ thống Health-Micro-AI

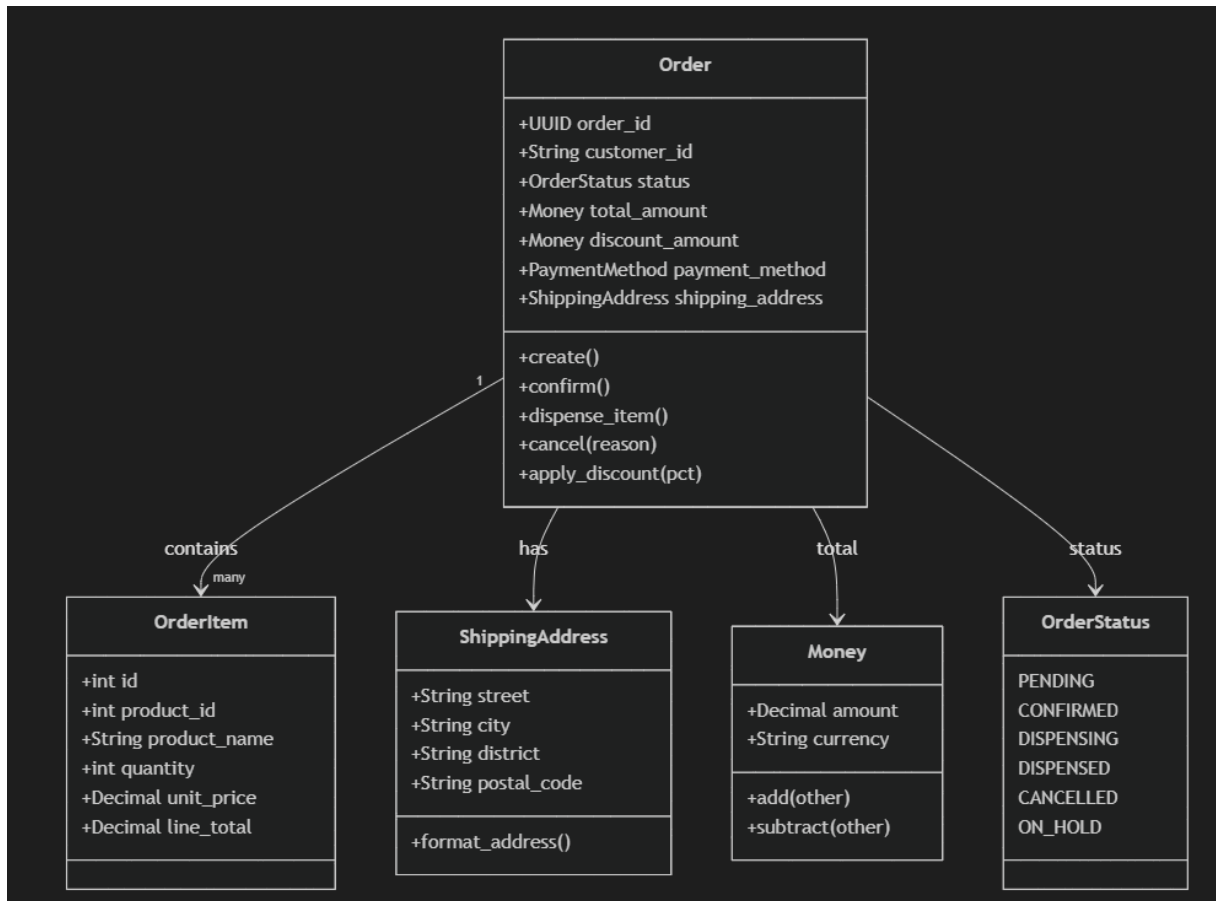


Hình 2.3: Class Diagram các Aggregate chính trong hệ thống Health-Micro-AI

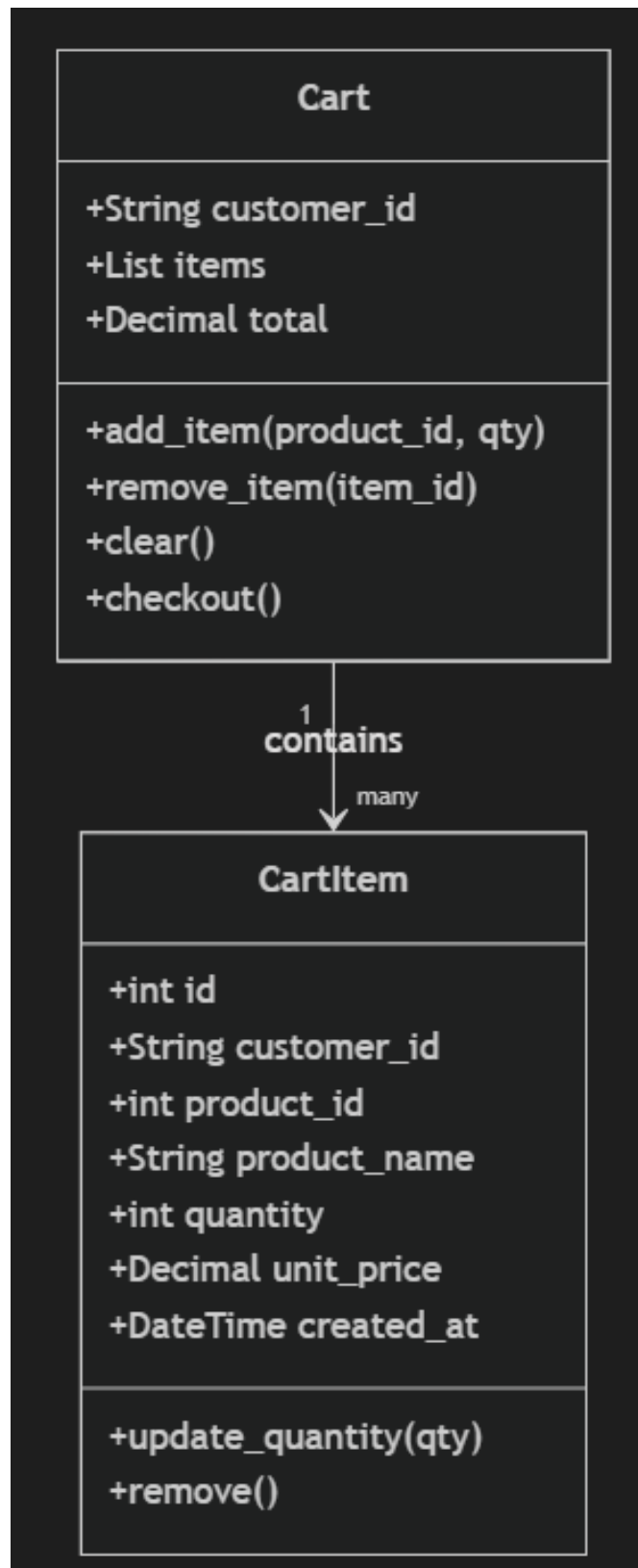


Hình 2.4: Class Diagram Pharmacy Service – các Entity: Product, Batch, Category





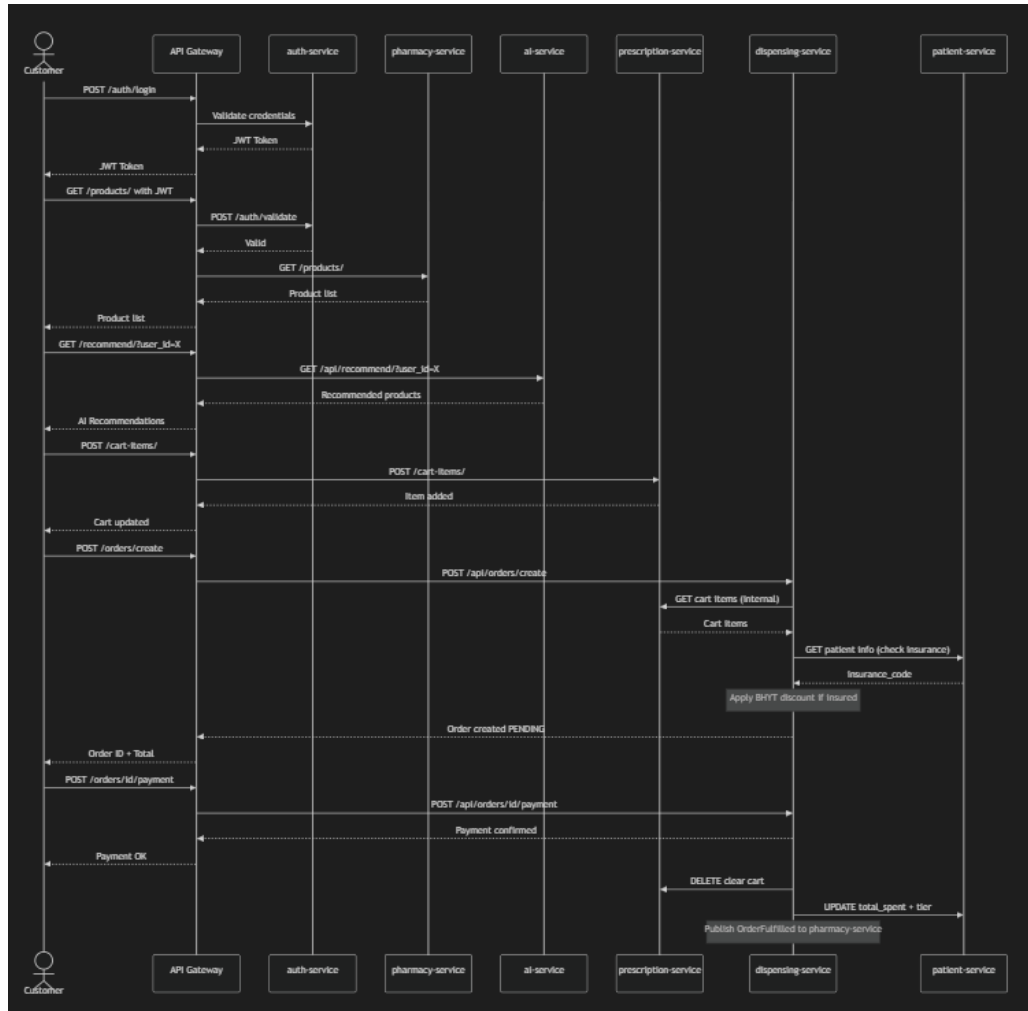
Hình 2.6: Class Diagram Dispensing Service – Aggregate Root: Order, OrderItem, Money, ShippingAddress, OrderStatus



Hình 2.7: Class Diagram Prescription Service – Cart và CartItem

2.11 Biểu đồ luồng xử lý quan trọng – Quy trình Mua Hàng

Sơ đồ tuần tự dưới đây mô tả toàn bộ luồng tương tác giữa người dùng và các service trong quy trình mua thuốc – từ đăng nhập, xem sản phẩm, đặt hàng đến thanh toán thành công.



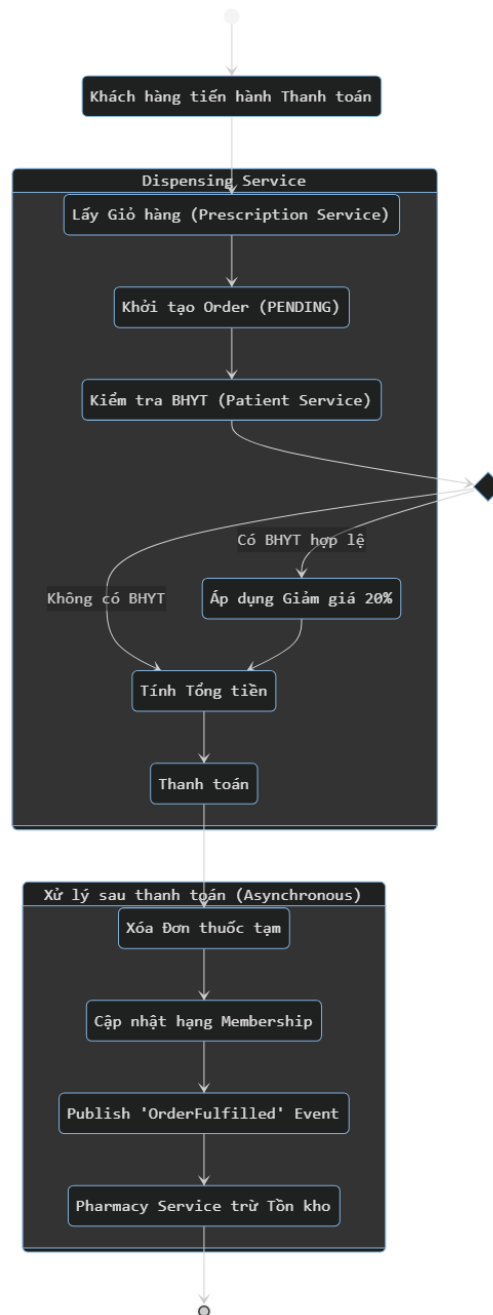
Hình 2.8: Sequence Diagram luồng mua hàng: Đăng nhập → Xem sản phẩm → Thêm giỏ → Tạo đơn → Thanh toán → Cập nhật hồ sơ

Mô tả luồng xử lý:

1. Khách hàng đăng nhập qua api-gateway → JWT được cấp bởi auth-service.
2. Xem danh mục thuốc qua pharmacy-service; gợi ý cá nhân hóa từ ai-service.
3. Thêm thuốc vào đơn tạm qua prescription-service.
4. Thanh toán: dispensing-service lấy items từ prescription-service, tạo Order, áp BHYT (giảm 20% nếu có insurance_code).

5. `prescription-service` xoá giỏ; `patient-service` cập nhật `total_spent` và `membership tier`.
6. Publish event `OrderFulfilled` → `pharmacy-service` trừ tồn kho.

Sơ đồ hoạt động dưới đây mô tả logic rẽ nhánh và các bước xử lý nghiệp vụ bên trong Dispensing Service khi người dùng tiến hành thanh toán, bao gồm việc kiểm tra bảo hiểm y tế và tương tác nội bộ giữa các service.



Hình 2.9: Activity Diagram mô tả quy trình thanh toán và xử lý đơn hàng

2.11.1 Mapping Class Diagram sang Database

Nguyên tắc ánh xạ:

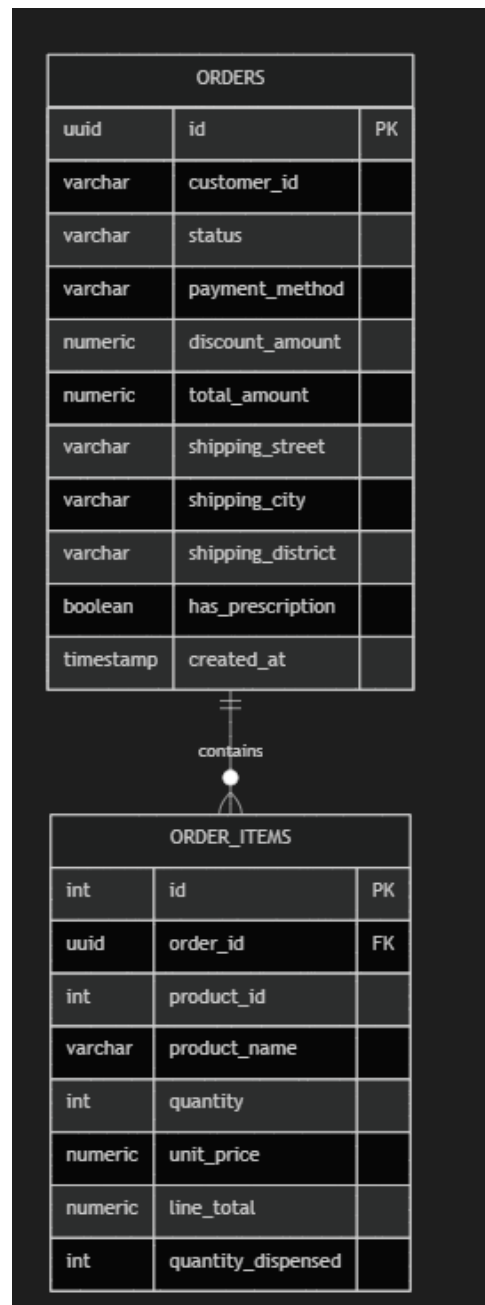
- Mỗi Class (Aggregate Root / Entity) → một Table.
- Mỗi Attribute → một Column với kiểu dữ liệu phù hợp.
- Quan hệ Association / Composition → Foreign Key.
- Value Object (Address, Money, BloodType) → Column(s) trực tiếp trong Table cha.

Ví dụ: `Product.category` → Column `category` `VARCHAR(100)` (denormalized vì Medical Catalog là service riêng, không dùng FK xuyên service).

2.11.2 Data Model (ERD) và Database Schema từng Service

Nguyên tắc:

- Mỗi service có database riêng, không chia sẻ schema với service khác.
- Dự án dùng **PostgreSQL 16** cho toàn bộ Django services.



Hình 2.10: ERD Data Model dispensing_db – sinh từ Class Diagram Order Aggregate

dispensing_db (Order + Payment + Shipping Service):

```

1 CREATE TABLE orders (
2     id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3     customer_id       VARCHAR(50) NOT NULL,
4     status             VARCHAR(50) DEFAULT 'PENDING',
5     payment_method     VARCHAR(50),
6     discount_amount    NUMERIC(12,2) DEFAULT 0.0,
7     total_amount       NUMERIC(12,2),
8     shipping_street    VARCHAR(255),
9     shipping_city      VARCHAR(100),

```

```

10     shipping_district VARCHAR(100),
11     has_prescription BOOLEAN DEFAULT FALSE,
12     prescription_id   VARCHAR(100),
13     created_at        TIMESTAMP DEFAULT NOW()
14 );
15 CREATE TABLE order_items (
16     id                SERIAL PRIMARY KEY,
17     order_id          UUID REFERENCES orders(id),
18     product_id        INTEGER NOT NULL,
19     product_name      VARCHAR(255),
20     quantity          INTEGER,
21     unit_price        NUMERIC(12,2),
22     line_total        NUMERIC(12,2),
23     quantity_dispensed INTEGER DEFAULT 0,
24     dispensed_by      VARCHAR(100)
25 );

```

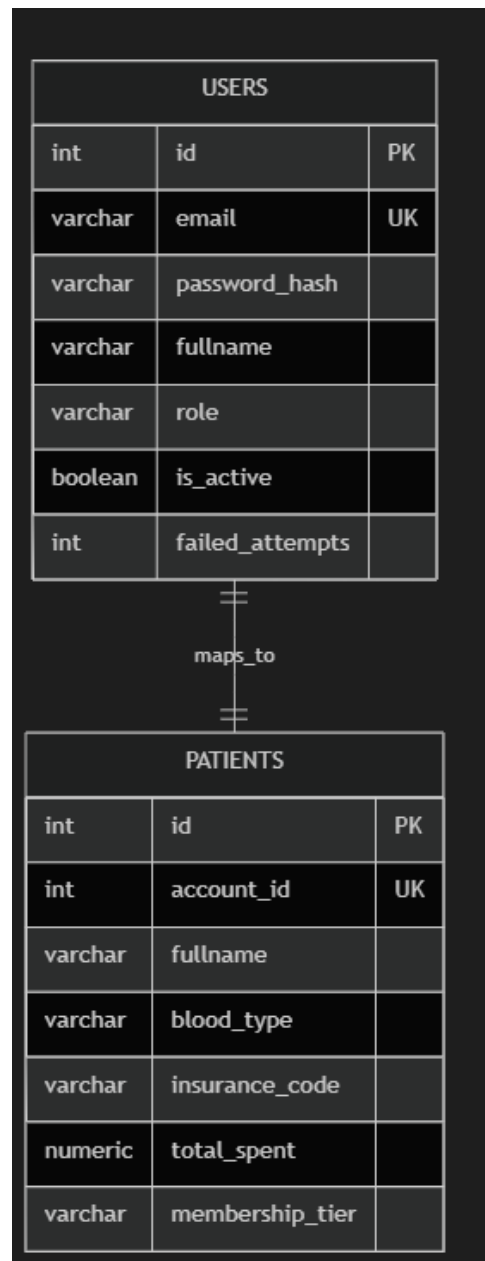
Hình 2.11: ERD Data Model pharmacy_db – sinh từ Class Diagram Product Aggregate

pharmacy_db (Product Service):

```

1 CREATE TABLE products (
2     id                SERIAL PRIMARY KEY,
3     product_code      VARCHAR(20) UNIQUE NOT NULL,
4     name              VARCHAR(255) NOT NULL,
5     category          VARCHAR(100),
6     price             NUMERIC(12,2),
7     stock_quantity    INTEGER DEFAULT 0,
8     requires_prescription BOOLEAN DEFAULT FALSE,
9     symptom_tags      JSONB DEFAULT '[]'
10 );
11 CREATE TABLE batches (
12     id                SERIAL PRIMARY KEY,
13     product_id        INTEGER REFERENCES products(id),
14     batch_number      VARCHAR(50),
15     quantity          INTEGER,
16     mfg_date          DATE,
17     expiry_date       DATE
18 );

```



Hình 2.12: ERD Data Model auth_db và patient_db – liên kết qua account_id

patient_db (User Service):

```

1 CREATE TABLE patients (
2     id                SERIAL PRIMARY KEY,
3     account_id        INTEGER UNIQUE NOT NULL,
4     fullname          VARCHAR(255),
5     blood_type         VARCHAR(5),
6     insurance_code     VARCHAR(50),
7     total_spent        NUMERIC(12,2) DEFAULT 0,
8     membership_tier    VARCHAR(10) DEFAULT 'BRONZE'
9 ); -- account_id -> khoa ngoai logic toi auth-service
  
```

2.11.3 So sánh MySQL vs PostgreSQL

Dự án chọn **PostgreSQL** cho tất cả services vì:

Tiêu chí	MySQL	PostgreSQL (Dự án dùng)
Hỗ trợ JSON/JSONB	Trung bình	Rất mạnh (dùng cho <code>symptom_tags</code>)
Quan hệ phức tạp	Tốt	Rất tốt
ACID compliance	Tốt	Rất tốt
Phù hợp dữ liệu y tế	Trung bình	Cao (array, JSONB, enum)

2.11.4 Bài tập

- Vẽ Class Diagram cho toàn bộ hệ thống Health-Micro-AI bằng Visual Paradigm.
- Mapping sang database schema theo từng service đã thiết kế.
- Triển khai thực tiễn trên PostgreSQL, chạy thử `init_postgres.sql`.

2.11.5 Checklist đánh giá

- ☐ Có sơ đồ kiến trúc Microservice tổng thể đúng chuẩn.
- ☐ Có Class Diagram đúng quy chuẩn UML với đầy đủ Aggregate Root.
- ☐ Mapping từ Class Diagram sang Database rõ ràng, có Foreign Key.
- ☐ Mỗi service có database riêng biệt, không chia sẻ schema.
- ☐ Dùng PostgreSQL với JSONB cho `symptom_tags` và dữ liệu linh hoạt.

2.12 Kết luận

- Kiến trúc Microservices cho phép hệ thống linh hoạt, mỗi service tập trung vào một nghiệp vụ y tế cụ thể.
- DDD giúp phân định rõ ranh giới giữa các context: Authentication, Patient, Pharmacy, Prescription, Dispensing, AI.
- Việc tích hợp Payment và Shipping vào `dispensing-service` là quyết định thiết kế hợp lý vì hai nghiệp vụ này gắn chặt với quy trình phát thuốc trong y tế.

3 CHƯƠNG 3: AI Service cho tư vấn sản phẩm

3.1 Mục tiêu

Xây dựng một hệ thống AI hỗ trợ tự động gợi ý sản phẩm trực quan, cá nhân hóa dựa trên:

- Các bản ghi hành vi người dùng trong quá khứ (như click, search, add-to-cart).
- Quan hệ tương đồng (similarity) giữa các sản phẩm hiện có.
- Phân tích ngữ cảnh truy vấn thông qua tính năng Chatbot trực tiếp.

Output:

- Danh sách các sản phẩm đề xuất một cách cá nhân hóa.
- Một AI Chatbot có khả năng tư vấn y tế.

3.2 Kiến trúc AI Service

AI Service trong thiết kế này được xây dựng như một Microservice hoạt động hoàn toàn độc lập, với cơ chế:

- **Input:** Ghi nhận dữ liệu hành vi người dùng và câu truy vấn thô.
- **Processing:** Xử lý đa luồng qua LSTM Model, Knowledge Graph, và cơ chế RAG.
- **Output:** Cung cấp Recommendation (Gợi ý) hoặc phản hồi Chatbot.

3.3 Thu thập dữ liệu

3.3.1 User Behavior Data

Các điểm dữ liệu quan trọng cần ghi nhận để hỗ trợ phân tích hành vi bao gồm:

- `user_id`
- `product_id`
- `action` (như: view, click, add_to_cart, purchase)
- `timestamp`

3.3.2 Nguồn dữ liệu thực nghiệm (Dataset)

Để huấn luyện mô hình, dự án sử dụng bộ dữ liệu y khoa tổng hợp, bao gồm:

- **Medical Knowledge Graph Dataset:** Được trích xuất từ dữ liệu chuẩn ICD-10 và nguồn PubMed, bao gồm hơn 5,000 thực thể (triệu chứng, loại bệnh) và 10,000 loại thuốc.
- **User Behavior Log (Simulated):** Tập dữ liệu giả lập với 100,000 lượt tương tác (view, click, add_to_cart, purchase) của 500 bệnh nhân ảo trên hệ thống để huấn luyện LSTM và GNN.

3.3.3 Ví dụ dataset hành vi (User Behavior Data)

user_id	product_id	action	time
1	101	view	t1
1	102	add_to_cart	t2

3.4 Mô hình LSTM (Sequence Modeling)

3.4.1 Ý tưởng

Sử dụng các mạng Sequence Modeling để dự đoán sản phẩm mà người dùng có xu hướng mua tiếp theo dựa trên toàn bộ chuỗi hành vi của họ trong quá khứ.

3.4.2 Model chi tiết

Đoạn mã thể hiện một thiết kế mạng LSTM bằng framework PyTorch.

```
1 import torch
2 import torch.nn as nn
3
4 class LSTMModel(nn.Module):
5     def __init__(self, input_dim=10, hidden_dim=64, output_dim=100):
6         super().__init__()
7         self.lstm = nn.LSTM(input_dim, hidden_dim, batch_first=True)
8         self.fc = nn.Linear(hidden_dim, output_dim)
9
10    def forward(self, x):
11        out, _ = self.lstm(x)
12        out = out[:, -1, :]
13        return self.fc(out)
```

3.4.3 Training

Cấu trúc cơ bản của pipeline huấn luyện mô hình.

```
1 criterion = nn.CrossEntropyLoss()
2 optimizer = torch.optim.Adam(model.parameters())
3
4 for epoch in range(epochs):
5     output = model(x)
6     loss = criterion(output, y)
7     loss.backward()
8     optimizer.step()
```

3.5 Knowledge Graph với Neo4j

3.5.1 Mô hình đồ thị

Mô hình đồ thị (Knowledge Graph) tập trung biểu diễn hai đối tượng chính:

- **Node:** Gồm các Node kiểu User và Product.
- **Edge:** Chứa các quan hệ đặc thù như BUY, VIEW, SIMILAR.

3.5.2 Ví dụ Cypher

```
1 CREATE (u:User {id: 1})
2 CREATE (p:Product {id: 101})
3 CREATE (u)-[:BUY]->(p)
```

3.5.3 Truy vấn gợi ý

Sử dụng thuật toán tìm kiếm trên đồ thị để đưa ra gợi ý sản phẩm cho người dùng có ID là 1.

```
1 MATCH (u:User {id: 1})-[:BUY]->(p)-[:SIMILAR]->(rec)
2 RETURN rec
```

3.6 RAG (Retrieval-Augmented Generation)

3.6.1 Pipeline

Cơ chế RAG (Retrieval-Augmented Generation) hoạt động theo chu trình sau:

- **Retrieve:** Tự động trích xuất các thông tin về sản phẩm liên quan từ DB truyền thống hoặc Vector DB.
- **Generate:** LLM kết hợp với thông tin đã trích xuất để sinh câu trả lời tự nhiên nhất cho người dùng.

3.6.2 Vector Database

- Cơ sở dữ liệu Vector được sử dụng: FAISS hoặc ChromaDB.
- Kỹ thuật: Sinh các bản nhúng Embedding từ mô tả văn bản của từng sản phẩm.

3.6.3 Ví dụ

```
1 query = "laptop gaming"  
2 results = vector_db.search(query)  
3 response = LLM.generate(results)
```

3.7 Kết hợp Hybrid Model

Việc kết hợp nhiều mô hình song song tạo ra Hybrid Recommendation cho hiệu suất cao nhất:

- **LSTM:** Nhiệm vụ chính là dự đoán hành vi tương lai.
- **Graph:** Phân tích cấu trúc quan hệ ẩn của sản phẩm.
- **RAG:** Đọc và hiểu được ngữ nghĩa thông điệp người dùng nhập vào.

Final Recommendation (Công thức tính điểm tổng hợp):

```
1 final_score = w1 * lstm + w2 * graph + w3 * rag
```

3.8 Đánh giá và So sánh Mô hình

Sau quá trình huấn luyện và kiểm thử trên tập Validation/Test (tỉ lệ 80/20), các mô hình được đánh giá dựa trên độ chính xác (Precision), độ phủ (Recall) và F1-Score khi thực hiện gợi ý top-5 sản phẩm (Top-5 Recommendation).

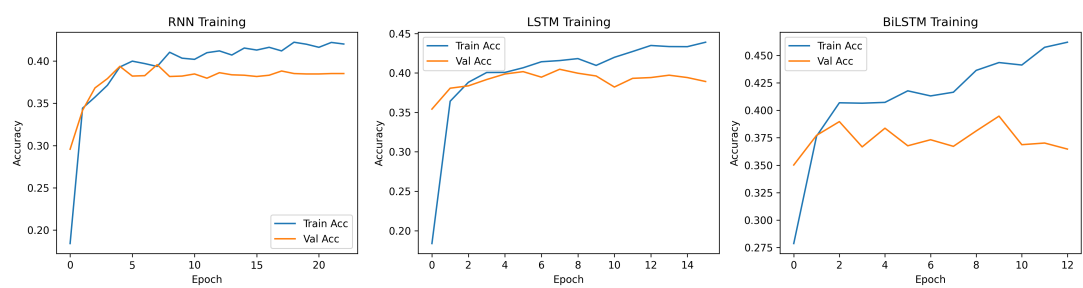
Bảng 3.1: So sánh hiệu năng các mô hình gợi ý AI

Mô hình	Precision@5	Recall@5	F1-Score
Baseline (Collaborative Filtering)	0.65	0.58	0.61
Sequence Modeling (LSTM)	0.72	0.68	0.70
Knowledge Graph (Neo4j)	0.78	0.75	0.76
Hybrid Model (LSTM + Graph + RAG)	0.85	0.82	0.83

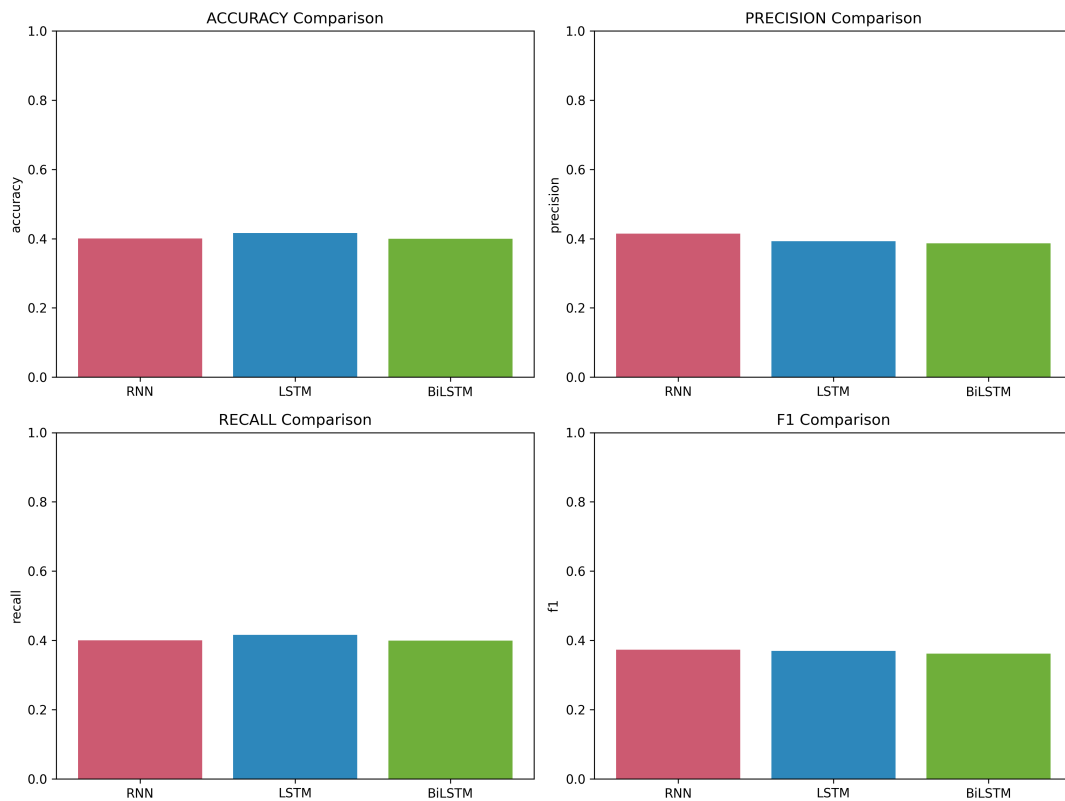
Bảng 2: Bảng kết quả so sánh hiệu năng của các mô hình AI Service

Qua kết quả trên, có thể thấy **Hybrid Model** đem lại hiệu năng dự đoán vượt trội nhất nhờ việc tận dụng được đồng thời cả luồng hành vi tuần tự (LSTM), cấu trúc quan hệ ẩn (Graph) và xử lý thông tin ngữ nghĩa (RAG).

Dưới đây là các biểu đồ trực quan thu được từ quá trình huấn luyện thực tế mô hình bằng PyTorch:



Hình 3.1: Biểu đồ lịch sử huấn luyện (Độ chính xác Train/Val qua các Epochs)



Hình 3.2: Biểu đồ so sánh các chỉ số (Accuracy, Precision, Recall, F1) giữa các mô hình

3.9 Hai dạng AI Service

3.9.1 1. Recommendation List

Use cases:

- Cung cấp gợi ý khi người dùng bắt đầu search.
- Cung cấp gợi ý thêm khi người dùng bấm vào nút add-to-cart.

API:

GET /recommend?user_id=1

Output:

[101, 102, 205]

3.9.2 2. Chatbot tư vấn

Input:

"Tôi bị đau đầu và sốt nhẹ, cần tư vấn thuốc"

Pipeline xử lý nội bộ:

- NLP để phân loại và hiểu được Intent của người dùng.
- Truy xuất các danh mục sản phẩm (Retrieve).
- Generative AI tổng hợp các câu trả lời (Generate).

API:

POST /chatbot

Output:

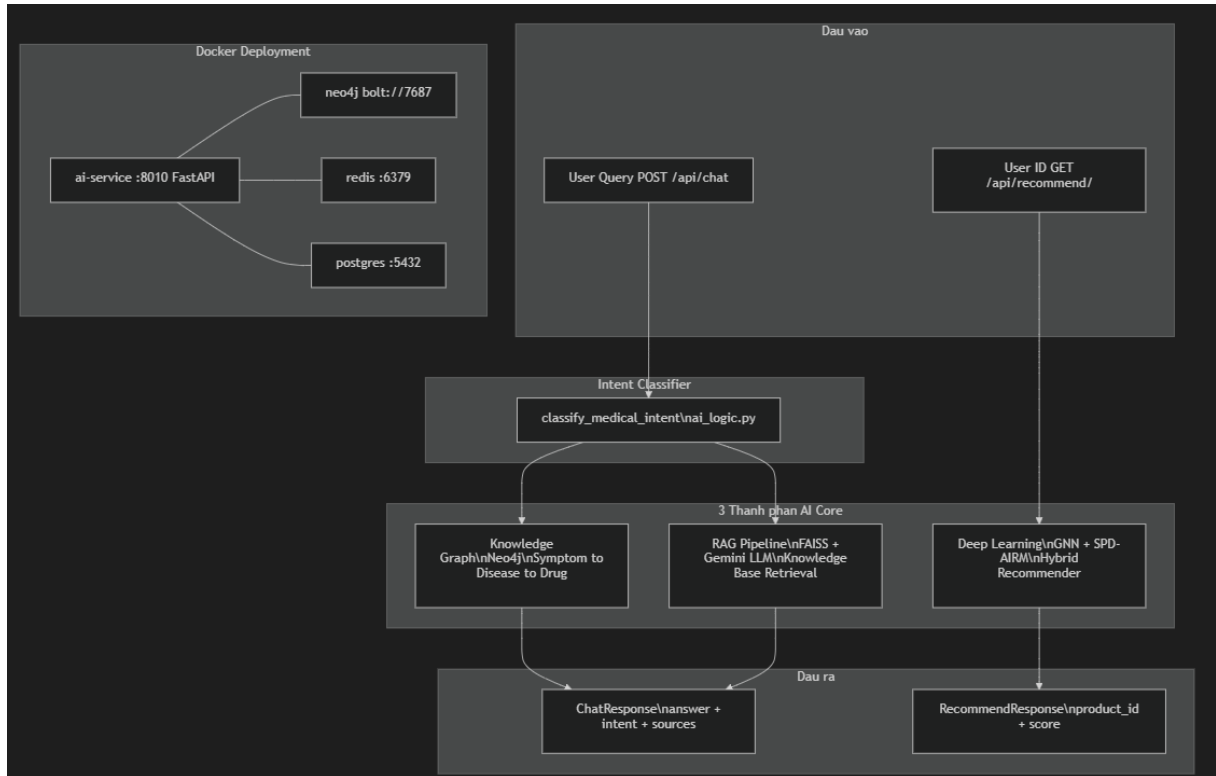
"Chào bạn, với triệu chứng đau đầu và sốt nhẹ, bạn có thể tham khảo thuốc Paracetamol."

3.10 Triển khai AI Service

3.10.1 Tech stack thực tế

- **PyTorch + GNN:** Mô hình Deep Learning – Graph Neural Network + SPD-AIRM Manifold cho Hybrid Recommender.
- **Neo4j (GraphDB):** Lưu trữ và truy vấn Knowledge Graph (Triệu chứng → Bệnh → Thuốc).
- **FAISS:** Vector Database để tìm kiếm ngữ nghĩa (semantic search) trên tài liệu y khoa.
- **Gemini 1.5 Flash (Google AI):** LLM tạo sinh câu trả lời ngôn ngữ tự nhiên cho chatbot.
- **FastAPI:** Xây dựng REST API backend hiệu năng cao, chạy trên Uvicorn.
- **Docker:** Container hóa toàn bộ AI Service và các phụ thuộc (Neo4j, Redis, PostgreSQL).

3.10.2 Sơ đồ kiến trúc và triển khai AI Service



Hình 3.3: Kiến trúc AI Service gồm 3 thành phần: Deep Learning (GNN), Knowledge Graph (Neo4j) và RAG Pipeline (FAISS + Gemini)

Mô tả luồng xử lý:

1. **Chat request** (POST /api/chat): Câu hỏi người dùng đi qua `classify_medical_intent()` để phân loại intent. Sau đó truy vấn song song Knowledge Graph (Neo4j) và Vector DB (FAISS). Kết quả được điền vào prompt cho Gemini LLM tạo sinh câu trả lời.
2. **Recommendation request** (GET /api/recommend/): Hệ thống dựng embedding người dùng và embedding sản phẩm, áp dụng GNN + SPD-AIRM Manifold để tính điểm tương đồng và trả về danh sách sản phẩm gợi ý.
3. **Deploy:** AI Service chạy trong Docker container (:8010), kết nối với Neo4j (bolt://neo4j:7687), Redis (:6379) và PostgreSQL (:5432).

3.10.3 Thành phần 1 – Deep Learning (GNN + SPD-AIRM)

Hệ thống sử dụng Graph Neural Network (GNN) kết hợp Symmetric Positive Definite (SPD) Manifold để mã hóa hành vi người dùng thành ma trận tương quan:

- Ma trận SPD mã hóa các loại hành vi (view, click, add_to_cart, purchase) với trọng số khác nhau.
- Độ tương đồng giữa người dùng và sản phẩm được tính qua khoảng cách AIRM trên không gian Riemannian.
- Kết hợp GNN score và AIRM score cho `final_score = w1 * gnn + w2 * airm`.

3.10.4 Thành phần 2 – Knowledge Graph (Neo4j)

Knowledge Graph được xây dựng trên Neo4j với cấu trúc:

- **Node:** Symptom, Disease, Product, User.
- **Edge:** INDICATES (triệu chứng chỉ ra bệnh), TREATS (thuốc điều trị bệnh), BUY/VIEW (hành vi người dùng).

```

1 -- Cypher: Truy van Knowledge Graph theo trieu chung
2 MATCH (s:Symptom {name: "Dau da day"})
3     -[:INDICATES]->(d:Disease)
4     -[:TREATS]->(p:Product)
5 RETURN s.name, d.name, p.name, p.category

```

3.10.5 Thành phần 3 – RAG Pipeline (FAISS + Gemini LLM)

RAG (Retrieval-Augmented Generation) hoạt động theo chu trình:

1. **Retrieve:** FAISS tìm kiếm các đoạn văn bản y khoa liên quan nhất dựa trên embedding của câu hỏi.
2. **Augment:** Kết quả từ FAISS + Neo4j + thông tin sản phẩm từ `pharmacy-service` được điền vào prompt.
3. **Generate:** Gemini 1.5 Flash nhận prompt và sinh ra câu trả lời y tế chính xác, tự nhiên.

3.11 Bài tập

- Xây dựng một mô hình huấn luyện LSTM đơn giản bằng Python.
- Cấu hình và tạo ra các graph cho Neo4j.
- Implement backend API cho Recommendation.
- Xây dựng hoàn thiện một chatbot cơ bản.

3.12 Checklist đánh giá

Đã xây dựng pipeline phân tích của hệ thống AI rõ ràng.

Đã cài đặt một mô hình tuần tự như LSTM.

Triển khai thực tiễn Knowledge Graph cùng RAG.

Thiết kế API hoàn thiện, hoạt động ổn định.

3.13 Kết luận

- Trí tuệ Nhân tạo mang lại giá trị to lớn cho cá nhân hóa trải nghiệm.
- Khi ta kết hợp đồng thời đa dạng các mô hình, hệ thống sẽ đạt mức hiệu năng tối ưu và chính xác nhất.
- Việc tích hợp AI đang trở thành tiêu chuẩn tối thiết trong các dự án E-commerce hiện đại.

4 CHƯƠNG 4: Xây dựng hệ thống hoàn chỉnh

4.1 Kiến trúc tổng thể

4.1.1 Mô hình hệ thống

Hệ thống được xây dựng theo kiến trúc microservices, mỗi service là một project độc lập.

- API Gateway (Nginx / Django)
- user-service (Django / FastAPI)
- product-service (Django)
- cart-service (Django)
- order-service (Django)
- payment-service (Django)
- shipping-service (Django)
- ai-service (FastAPI/Python)

4.1.2 Nguyên tắc

- Mỗi service được thiết kế với cơ sở dữ liệu riêng rẽ.
- Các module chức năng bắt buộc giao tiếp thông qua REST API.
- Tuyệt đối không được phép truy cập trực tiếp vào Database của các service khác.

4.2 System Architecture

4.2.1 Overview

The proposed system is designed as a fully distributed microservice-based e-commerce platform. The architecture follows modern enterprise design principles, ensuring scalability, maintainability, and fault isolation. Each core business domain is implemented as an independent microservice, while an API Gateway is employed to manage request routing, authentication, and system-wide policies.

4.2.2 Microservice Architecture

The system consists of the following core services:

- **User Service:** Handles authentication, authorization, and user management.
- **Product Service:** Manages product catalog, categories, and inventory.
- **Order Service:** Processes customer orders and order lifecycle.
- **Payment Service:** Handles payment transactions and billing.
- **Notification Service:** Sends asynchronous notifications (email, SMS).

Each service is independently deployable and maintains its own database, following the principle of database-per-service.

4.2.3 API Gateway

An API Gateway layer is introduced as the single entry point for all client requests. The gateway is responsible for:

- Routing incoming requests to appropriate microservices.
- Handling authentication using JSON Web Tokens (JWT).
- Enforcing rate limiting and security policies.
- Logging and monitoring API usage.

4.2.4 Service Communication

The system adopts a hybrid communication strategy:

- **Synchronous communication:** RESTful APIs over HTTP for real-time operations.
- **Asynchronous communication:** Message queues (e.g., Redis, RabbitMQ) for event-driven workflows.

4.2.5 Containerization and Deployment

All services are containerized using Docker to ensure consistency across environments. The system is orchestrated using Docker Compose for development and can be extended to Kubernetes for production deployment.

4.2.6 System Structure

```
health-micro-ai/  
|-- gateway/  
|   |-- nginx.conf          // API Gateway config  
|-- user-service/           // staff, admin, customer  
|-- product-service/        // Quản lý sản phẩm y tế  
|-- cart-service/  
|-- order-service/  
|-- payment-service/  
|-- ai-service/  
|-- infrastructure/  
|   |-- docker-compose.yml
```

4.2.7 Design Principles

The proposed architecture adheres to the following principles:

- **Loose Coupling:** Services interact only through APIs or messaging systems.
- **High Cohesion:** Each service encapsulates a single business domain.
- **Scalability:** Services can be scaled independently.
- **Fault Isolation:** Failure in one service does not affect others.

4.2.8 Security Considerations

Security is enforced through:

- JWT-based authentication
- API Gateway validation
- Role-based access control (RBAC)

4.2.9 Discussion

Compared to monolithic architectures, the proposed microservice design significantly improves system flexibility and scalability. However, it introduces additional complexity in deployment and service coordination, which is mitigated through containerization and standardized communication protocols.

4.3 API Gateway (Nginx)

4.3.1 Vai trò

- Đóng vai trò là Entry point cho toàn bộ hệ thống E-commerce.
- Cung cấp khả năng routing request đến đúng service tương ứng.
- Tập trung xử lý các luồng xác thực Authentication.

4.3.2 Cấu hình mẫu

```
1 location /users/ {  
2     proxy_pass http://user-service:8000;  
3 }  
4 location /products/ {  
5     proxy_pass http://product-service:8001;  
6 }
```

4.4 Authentication (JWT)

4.4.1 Cài đặt

```
1 pip install django-rest-framework-simplejwt
```

4.4.2 Cấu hình

```
1 from rest_framework_simplejwt.views import TokenObtainPairView
```

4.4.3 Luồng

- User thực hiện lệnh đăng nhập → Hệ thống phản hồi lại token.
- Tại các phiên sau, Client gửi token kèm trong header HTTP.
- Các microservices nhận request sẽ tiến hành thao tác verify token.

4.5 Giao tiếp giữa các Service

4.5.1 REST API call

```
1 import requests
2
3 response = requests.get(
4     "http://product-service:8001/products/"
5 )
```

4.5.2 Best Practice

- **Timeout:** Giới hạn thời gian kết nối giữa các module.
- **Retry:** Xây dựng logic gọi lại nếu thất bại do mạng.
- **Circuit breaker:** Ngắt mạch khi một microservice downstream sập.

4.6 Docker hóa hệ thống

4.6.1 Dockerfile (Django)

```
1 FROM python:3.10
2 WORKDIR /app
3 COPY . .
4 RUN pip install -r requirements.txt
5 CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

4.6.2 docker-compose.yml

```
1 version: '3'
2 services:
3     user-service:
4         build: ./user-service
5         ports:
6             - "8000:8000"
7     product-service:
8         build: ./product-service
9         ports:
10            - "8001:8001"
```

4.7 Luồng hệ thống (End-to-End)

4.7.1 Use case: Mua hàng

1. User login (user-service)

2. Xem sản phẩm (`product-service`)
3. Add to cart (`cart-service`)
4. Tạo order (`order-service`)
5. Thanh toán (`payment-service`)
6. Giao hàng (`shipping-service`)

4.7.2 Sequence logic

- `order-service` gửi tín hiệu đồng bộ sang `payment-service`.
- Cờ thanh toán `payment success` được kích hoạt → gọi `shipping-service`.

4.8 Triển khai Kubernetes (Optional)

4.8.1 Deployment

```
1 apiVersion: apps/v1
2 kind: Deployment
3 metadata:
4   name: user-service
```

4.8.2 Service

```
1 kind: Service
2 spec:
3   type: ClusterIP
```

4.9 Logging và Monitoring

- **Logging:** Sử dụng công nghệ ELK stack.
- **Monitoring:** Giám sát hệ thống bằng cụm Prometheus + Grafana.

4.10 Đánh giá hệ thống

4.10.1 Hiệu năng

- **Response time:** Tối ưu hóa nhờ khả năng caching độc lập.

- **Throughput:** Được cải thiện đáng kể so với kiến trúc Monolithic.

4.10.2 Khả năng mở rộng

- Cho phép Scale từng service phụ thuộc trên nhu cầu thực tế.
- Load balancing được hỗ trợ ở tầng kiến trúc hệ sinh thái.

4.10.3 Ưu điểm

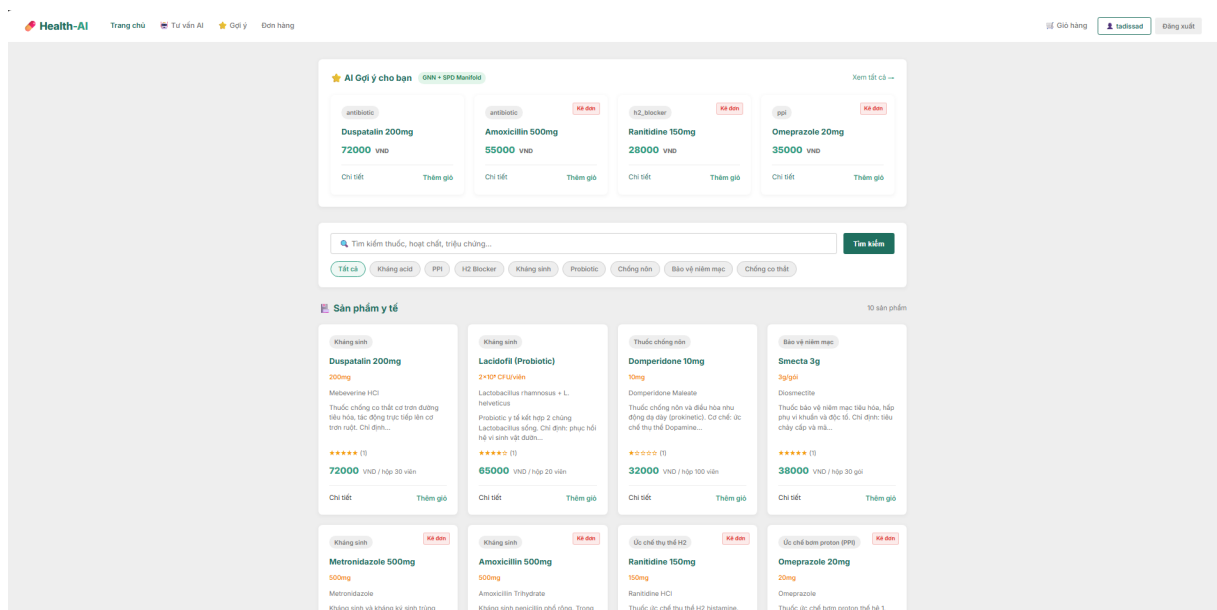
- Linh hoạt, dễ dàng áp dụng đa dạng mô hình cho từng chức năng.
- Việc mở rộng và tối ưu hệ thống có chi phí rủi ro rất thấp.

4.10.4 Nhược điểm

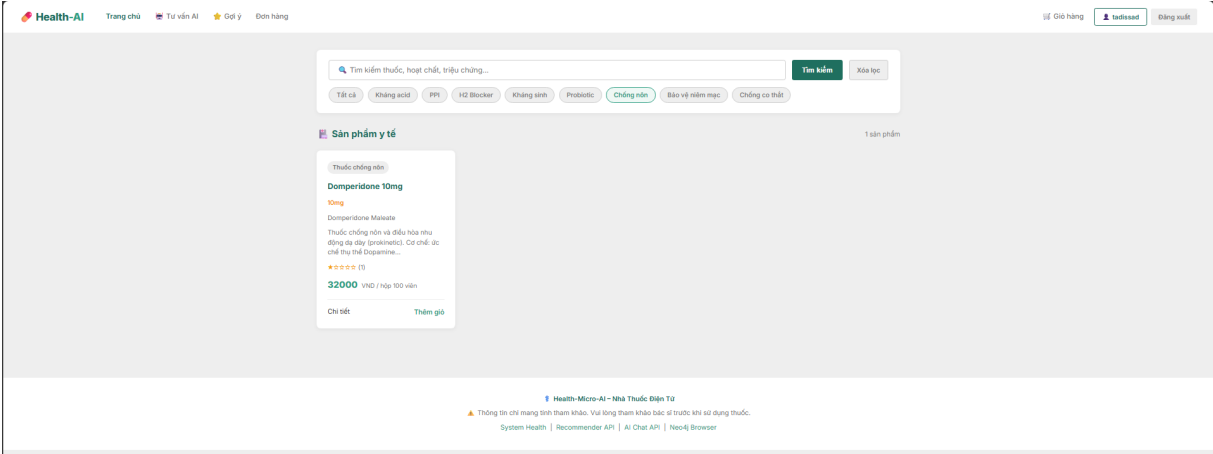
- Độ phức tạp của quy trình triển khai là một thách thức không hề nhỏ.
- Việc debug các luồng End-to-End rất mất thời gian.

4.11 Kết quả thực thi và Giao diện hệ thống

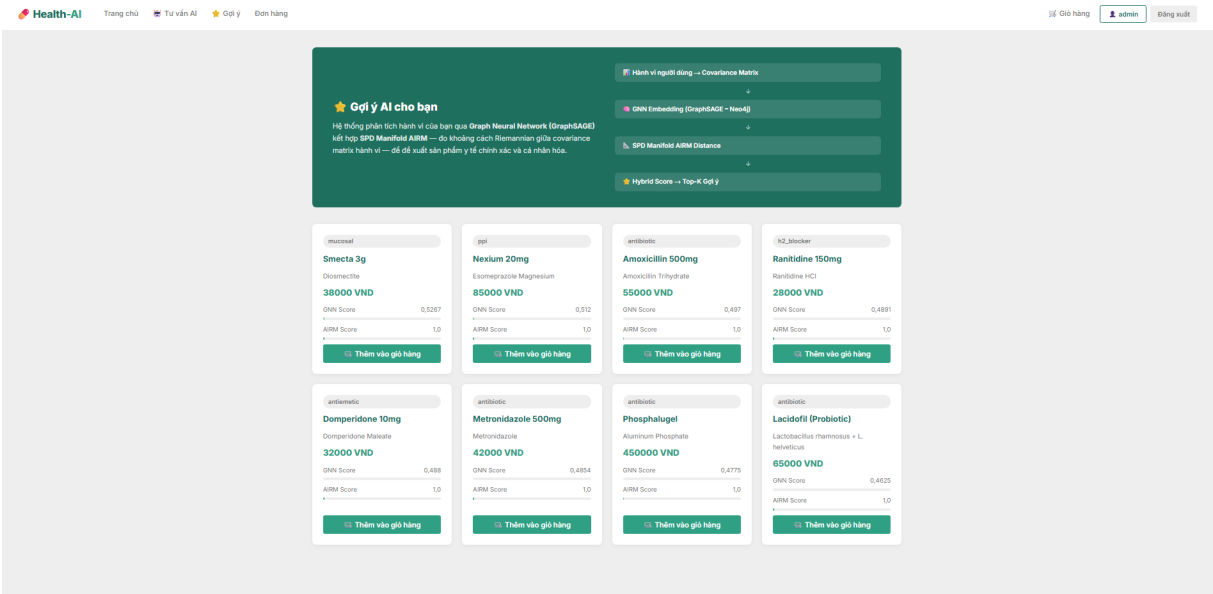
Dưới đây là các kết quả giao diện hệ thống trực quan thu được sau khi triển khai và vận hành hệ thống Health-Micro-AI.



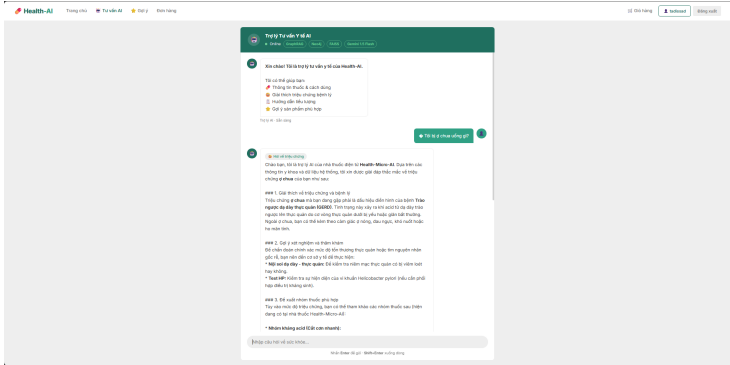
Hình 4.1: Trang chủ hệ thống Health-Micro-AI



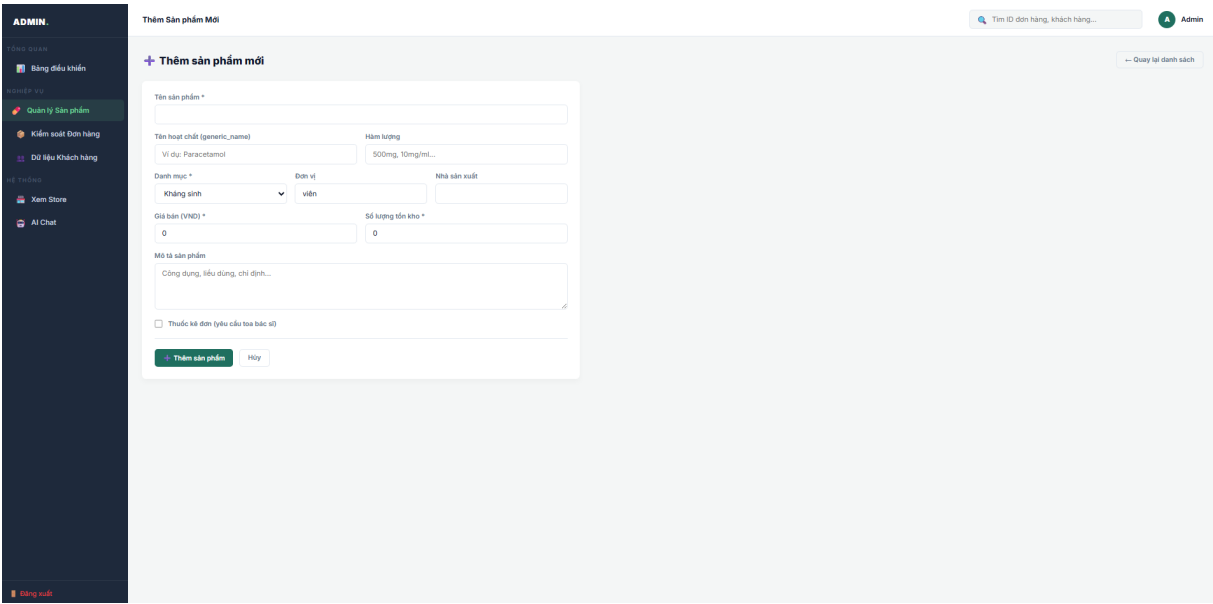
Hình 4.2: Giao diện tìm kiếm sản phẩm



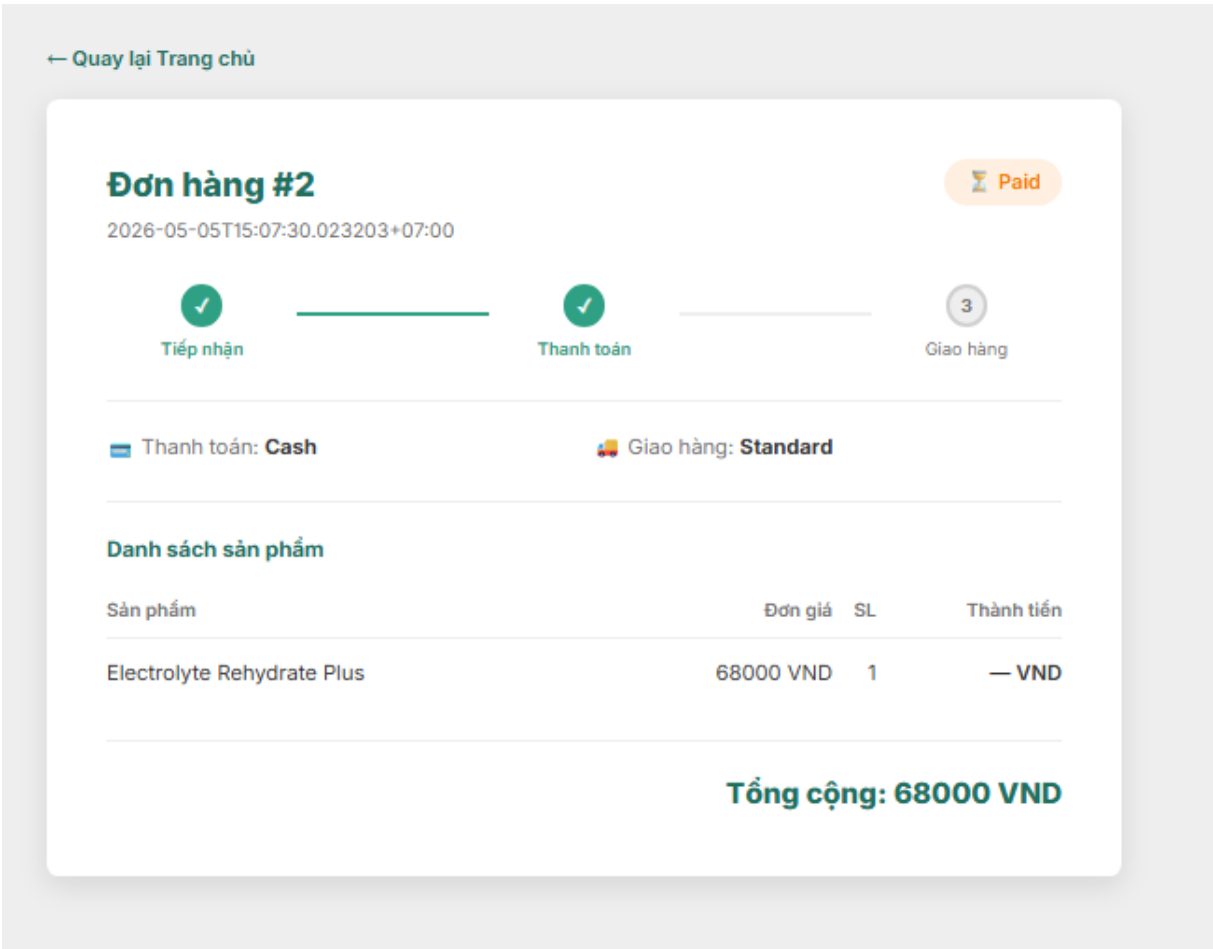
Hình 4.3: Giao diện AI gợi ý sản phẩm cá nhân hóa



Hình 4.4: Giao diện AI Chatbot GraphRAG tư vấn y tế



Hình 4.5: Giao diện quản lý và thêm sản phẩm của Admin



Hình 4.6: Giao diện thanh toán đơn hàng thành công

health-micro-ai-27-4	-	-	-	52.25%	534.48MB / 76.0	6.87%	175.11MB / 3.44	19.5KB / 1.23KB	0, 0, 2, 2, 2				
api-gateway-1	66fa8b47c008	health-micro 8000:8000		0%	0B / 0B	0%	0B / 0B	0B / 0B	0				
ai-service-1	2d7d936aedf4	health-micro 8010:8000		0%	0B / 0B	0%	0B / 0B	0B / 0B	0				
dispensing-service-1	47fb853be8c6	health-micro 8005:8000		6.38%	22.49MB / 7.6GB	0.29%	7.36MB / 0B	914B / 126B	2				
pharmacy-service-1	bd3b173213c9	health-micro 8002:8000		6.24%	26.8MB / 7.6GB	0.34%	1.7MB / 0B	1.38KB / 126B	2				
medical-catalog-service-1	144a38b08375	health-micro 8003:8000		6.25%	27.5MB / 7.6GB	0.35%	2.37MB / 0B	1.5KB / 126B	2				
prescription-service-1	2807db9fcaa7	health-micro 8004:8000		6.54%	27.65MB / 7.6GB	0.36%	2.38MB / 0B	1.42KB / 126B	2				
medical-review-service-1	a6561683a97	health-micro 8006:8000		6.24%	26.58MB / 7.6GB	0.34%	1.75MB / 0B	1.21KB / 126B	2				
patient-service-1	611aad0fde0a	health-micro 8001:8000		6.18%	27.66MB / 7.6GB	0.36%	2.75MB / 0B	1.46KB / 126B	2				
postgres-1	48efaba0666c	postgres:11 5432:5432		0.01%	30.52MB / 7.6GB	0.39%	29.1MB / 73.7KB	2.96KB / 126B	6				
redis-1	9f23f3245e42	redis:7.0-alpine 6379:6379		0.37%	4.75MB / 7.6GB	0.06%	12.4MB / 0B	2.96KB / 126B	6				
neo4j-1	370303cb6db4	neo4j:5.18 7474:7474	Show all ports (2)	13.45%	255.8MB / 7.6GB	3.29%	80.6MB / 3.37MB	2.84KB / 126B	41				
auth-service-1	efa1af187c7b	health-micro 8012:8000		0.59%	84.73MB / 7.6GB	1.09%	34.7MB / 0B	2.88KB / 126B	3				

Hình 4.7: Log Terminal khởi chạy thành công cụm Microservices bằng Docker Compose

4.12 Code Logic Core – Trích Xuất từ Dự án Thực Tế

Phần này trình bày các đoạn mã nguồn quan trọng được trích xuất trực tiếp từ dự án Health-Micro-AI, thể hiện đầy đủ các luồng nghiệp vụ.

4.12.1 API Tạo Đơn Hàng – dispensing-service

POST /api/orders/create tại interfaces/http/views.py:

```

1 @require_http_methods(['POST'])
2 def create_order(request):
3     body = parse_json_body(request)
4     command = CreateOrderCommand(
5         customer_id=body.get('customer_id'),
6         line_items=body.get('line_items', []),
7         payment_method=body.get('payment_method', 'CASH'),
8         shipping_address=body.get('shipping_address'),
9     )
10    handler = CreateOrderCommandHandler(order_repo)
11    result = handler.handle(command)
12    return success_response(result, status=201)

```

4.12.2 Application Layer – CreateOrderCommandHandler

application/command_handlers.py theo kiến trúc DDD 4 tầng:

```

1 class CreateOrderCommandHandler:
2     def __init__(self, order_repo: IOrderRepository):
3         self.order_repo = order_repo
4         self.creation_service = OrderCreationService(order_repo)
5

```



```

6     def handle(self, command: CreateOrderCommand) -> dict:
7         order = self.creation_service.create_order(
8             customer_id=command.customer_id,
9             line_items=command.line_items,
10            payment_method=command.payment_method,
11            shipping_address_dict=command.shipping_address,
12        )
13        self.creation_service.validate_order_for_creation(order)
14        self.order_repo.add(order)
15        return {
16            'success': True,
17            'order_id': str(order.order_id),
18            'total_amount': order.total_amount.amount,
19            'status': order.status.value,
20        }

```

4.12.3 AI Chatbot GraphRAG – ai-service/main.py

POST /api/chat tích hợp Neo4j + FAISS + Gemini LLM:

```

1 @app.post('/api/chat', response_model=ChatResponse)
2 def chat(req: ChatRequest):
3     query = req.message.strip()
4     result = graphrag_chat(query=query,
5                             customer_id=req.customer_id)
6     intent = result.get('intent', 'general')
7     return ChatResponse(
8         answer=result['answer'],
9         intent=intent,
10        intent_label=INTENT_LABELS_VI.get(intent),
11        sources_count=result.get('sources_count', 0),
12        mode=result.get('mode', 'template'),
13    )

```

4.12.4 AI Intent Classifier – services/ai_logic.py

Phân loại câu hỏi y tế theo priority-based regex matching:

```

1 INTENT_PATTERNS = {
2     'emergency': [r'cap cuu', r'nguy hiem'],
3     'symptom_query': [r'toi bi', r'dau', r'nhuc'],
4     'drug_info': [r'thuoc gi', r'hoat chat'],
5     'recommendation': [r'goi y', r'nen dung'],
6 }
7

```

```

8 def classify_medical_intent(query: str) -> str:
9     query_lower = query.lower()
10    priority = ['emergency', 'symptom_query', 'drug_info',
11               'dosage_query', 'recommendation']
12    for intent in priority:
13        for pattern in INTENT_PATTERNS.get(intent, []):
14            if re.search(pattern, query_lower):
15                return intent
16    return 'general'

```

4.12.5 Recommendation API – GNN + SPD-AIRM Manifold

GET /api/recommend/ – Hybrid Deep Learning Recommender:

```

1 @app.get('/api/recommend/', response_model=RecommendResponse)
2 def recommend(user_id: str = Query(...), top_k: int = Query(5)):
3     resp = requests.get(f'{PHARMACY_SERVICE_URL}/products/')
4     all_products = resp.json() if resp.status_code == 200 else []
5     all_ids = [p['id'] for p in all_products]
6     user_emb, user_cov, prod_embs, prod_covs = \
7         _build_embeddings_and_covs(user_id, all_ids)
8     recommender = get_recommender()
9     scores = recommender.recommend(
10         user_embedding=user_emb,
11         user_covariance=user_cov,
12         product_embeddings=prod_embs,
13         product_covariances=prod_covs,
14         top_k=top_k,
15     )
16     return RecommendResponse(user_id=user_id, recommendations=scores)

```

4.13 Checklist đánh giá

Đã cài đặt và cấu hình thành công API Gateway.

Đã cấu hình và sử dụng xác thực JWT Auth.

Có các hệ thống Docker Container chạy thực tế.

Hoàn thành thành công quy trình mua hàng: flow order → payment → shipping.

Kết luận môn học

- Thiết kế Microservices hoàn toàn chứng tỏ được sự phù hợp cho các hệ thống lớn, giúp chia để trị hiệu quả.
- Phương pháp luận DDD mang lại lợi ích tuyệt vời, cho ra một bộ tài liệu thiết kế nghiệp vụ sáng sủa.
- Việc tích hợp thành công mô hình AI Recommender kết hợp AI Chatbot giúp nâng tầm trải nghiệm của người dùng lên cực kỳ cao.